

KernelEvolve: Scaling Agentic Kernel Coding for Heterogeneous AI Accelerators at Meta

KernelEvolve Team, Meta Platforms

Making deep learning recommendation model (DLRM) training and inference fast and efficient is important. However, this presents three key system challenges – model architecture diversity, kernel primitive diversity, and hardware generation and architecture heterogeneity. The combination of the three diversity dimensions leads to a complex optimization space.

This paper presents KernelEvolve – an agentic kernel coding framework – to tackle heterogeneity at-scale for DLRM training and inference. KernelEvolve is designed to take kernel specifications as input and automate the process of kernel generation and optimization for recommendation model across heterogeneous hardware architectures through multiple programming abstractions, including Triton, CuTe DSL, and low-level hardware diagnostic languages, spanning the full hardware-software optimization stack. The kernel optimization process is described as graph-based search with selection policy, universal operator, fitness function, and termination rule, dynamically adapts to runtime execution context through retrieval-augmented prompt synthesis. The system integrates a persistent knowledge base encoding hardware-specific constraints for heterogeneous AI accelerators, enabling effective kernel generation even for proprietary architectures absent from LLM training corpora.

We designed, implemented, and deployed KernelEvolve to optimize a wide variety of production recommendation models across generations of NVIDIA and AMD GPUs, as well as Meta’s latest-generation AI accelerators (MTIA v3). We validate KernelEvolve on the publicly-available KernelBench suite, achieving 100% pass rate on all 250 problems across three difficulty levels, and 160 PyTorch ATen operators across three heterogeneous hardware platforms, demonstrating 100% correctness over all 480 operator-platform configurations. KernelEvolve reduces development time from weeks to hours and achieves substantial performance improvements by up to 17 times over PyTorch baselines across diverse production use cases and for heterogeneous AI systems at-scale. Beyond performance efficiency improvements, KernelEvolve significantly mitigates the programmability barrier for new AI hardware by enabling automated kernel generation for proprietary accelerators. We hope the insights and deployment experience presented in this paper will shed new light on the design of AI systems and optimization at-scale.

Date: July 8, 2026

ISCA 2026 Paper: https://gangliao.me/assets/pdf/kernelevolve_isca26_paper.pdf

Correspondence: Gang Liao: gangliao@meta.com, Carole-Jean Wu: carolejeanwu@meta.com, Gaoxiang Liu: gaoxiang@meta.com



Contents

1	Introduction	3
1.1	The Curse of Dimensionality	3
2	Background	8
2.1	Meta’s Recommendation Model	8
2.2	Data Preprocessing Operations	9
3	System Architecture	11
3.1	Tree Search and State Machine	12
3.2	Agentic Retrieval & Self-Managed Context	13
3.2.1	Deep Search Sub-Agent	14
3.2.2	Context Memory Sub-Agent	15
3.2.3	MTIA Knowledge Injection	16
3.3	File and Code Search	18
3.4	Evaluation and Tooling Framework	18
3.4.1	Kernel Output Format	18
3.4.2	AI Hardware Interpreters	19
3.4.3	Evaluation Code Generation	20
3.4.4	Unified Profiling: Triton MPP	21
3.4.5	Agentic Debugging in JIT Flow	22
3.4.6	Kernel Evaluation on FaaS	22
4	OSS Operator Evaluation	23
5	Monetization Case Study	24
5.1	Convolutional Transformer	24
5.2	Convolution on Heterogeneous Hardware	26
5.3	WuKong and InterFormer: Kernel Fusion	29
5.3.1	Optimized FM in WuKong	29
5.3.2	PFFN in InterFormer	30
5.4	Data Preprocessing Kernels on MTIA	31
5.4.1	MapId Transform	32
5.4.2	MergeBucketizedDense Transform	34
5.4.3	Summary	36
5.5	Sequence Learning: Batch Event Truncate	37
6	Future Directions	38
7	Related Work	39
8	Conclusion	40
9	Contributors	41
10	Acknowledgments	41
A	1D Convolution	47

1 Introduction

Modern online advertising systems demand massive-scale machine learning inference under stringent sub-second latency constraints. At Meta’s operational scale – serving more than hundreds of trillions of ad-ranking inferences daily across global data centers consuming hundreds of megawatts – the infrastructure deploys an extensive ensemble of multi-stage ranking models (retrieval, early-stage ranking, and second-stage ranking, often exceeding 1500 distinct models) to address diverse prediction tasks. These models execute across a heterogeneous fleet of AI accelerators, including Meta Training and Inference Accelerator (MTIA), AMD GPUs, and NVIDIA hardware. Within this ecosystem, computational kernels—the low-level primitive functions implementing tensor operations, data transformations, and feature engineering—constitute a dual bottleneck affecting both training and serving architecture.

From a performance perspective, ads serving operates under strict real-time constraints where microsecond-level kernel efficiency directly impacts user experience and revenue generation, as well as Meta’s Total Cost of Ownership (TCO). The economic implications are substantial: marginal kernel-level performance improvements translate to multi-million dollar reductions in infrastructure operating costs while simultaneously enhancing user engagement metrics that correlate directly with advertising revenue. From an architectural perspective, kernel coverage on target hardware fundamentally determines system deployment viability. When critical operators lack native implementations on AI accelerators, production systems may architect disaggregated serving topologies where missing operators execute on separate CPU tiers, or halt deployment until kernels are implemented. This fragmentation introduces cross-node network overhead (adding millisecond-scale latency), serialization costs, reduced reliability from cascading failures. Moreover, incomplete kernel coverage blocks model launches, directly constraining algorithmic innovation and competitive differentiation. This confluence of latency sensitivity, architectural constraints, and massive deployment scale elevates kernel optimization to a first-order system imperative for ads model serving infrastructure.

1.1 The Curse of Dimensionality

The computational landscape of production ads ranking systems exhibits unprecedented complexity across three critical dimensions: model architecture diversity, kernel primitive diversity and hardware generation and architecture heterogeneity. This combinatorial explosion creates a fundamental scalability crisis that manual kernel development approaches cannot address.

Hardware Diversity Across Vendors and Generations. Production deployment spans heterogeneous accelerators including NVIDIA GPUs, AMD GPUs, and Meta’s custom MTIA chips [Meta 2024] in Figure 1—each exhibiting distinct architectural characteristics that prevent direct kernel portability.

(1) Memory hierarchy heterogeneity: Architectures differ fundamentally in cache organization and bandwidth characteristics. NVIDIA architectures employ multi-level cache hierarchies with tens of megabytes of L2 cache, AMD architectures feature large Infinity Cache structures serving as shared L3-equivalent storage, while MTIA architectures implement custom on-chip SRAM subsystems optimized for recommendation inference patterns with distinct on-chip and off-chip bandwidth profiles. These differences necessitate architecture-specific memory access patterns and tiling strategies.

(2) Programming model fragmentation: Each platform exposes hardware through incompatible abstractions—CUDA’s thread-block model, Triton’s tile-based DSL [Tillet et al. 2019], AMD’s ROCm/HIP extensions, CuTe’s layout algebra for NVIDIA Hopper [NVIDIA 2025], MTIA’s C++ kernel DSL, and emerging frameworks like TileLang/TLX/Gluon [Wang et al. 2025c, Meta 2025b, OpenAI 2025]. These abstractions differ not merely in syntax but in fundamental execution models—from Triton’s automatic memory coalescing to CUDA’s explicit shared memory management—necessitating complete algorithmic restructuring rather than syntactic translation.

(3) Generational architectural discontinuities: Even within a single vendor family, generational transitions introduce fundamentally different execution models. For example, NVIDIA’s Ampere-to-Hopper evolution introduced the Tensor Memory Accelerator (TMA) for asynchronous bulk tensor transfers between global and shared memory, added a new 128-thread warp-group execution model to support WGMMMA tensor operations, and exposed multiple asynchronous execution pipelines using mbarriers and producer-consumer

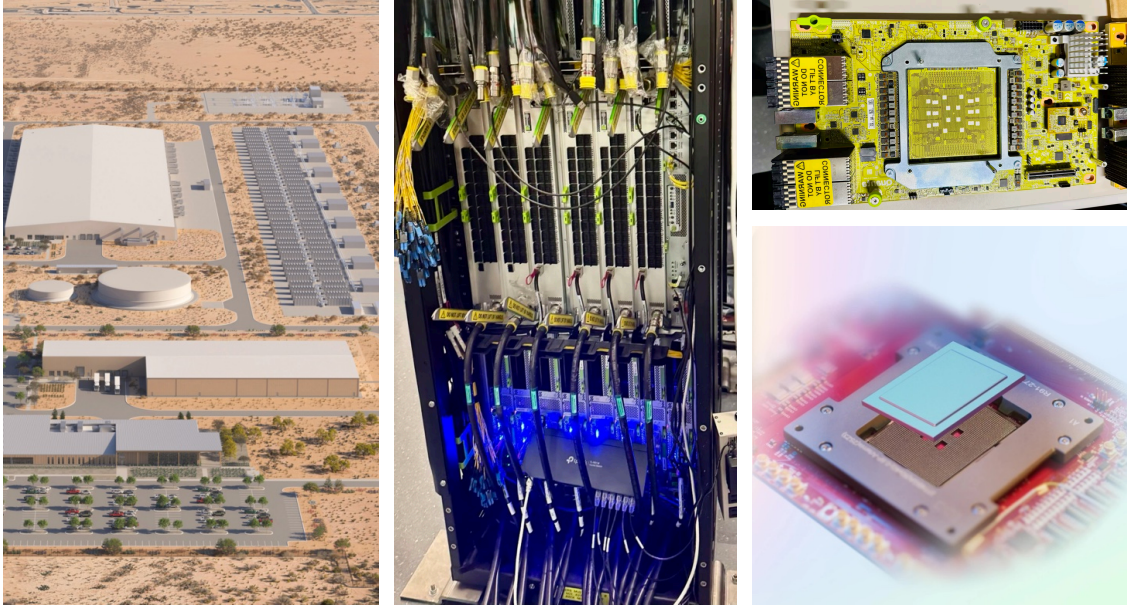


Figure 1 Meta’s MTIA (Meta Training and Inference Accelerator) is a custom-designed chip optimized for AI workloads. The figure illustrates the MTIA hardware from four perspectives: its integration within a data center or server farm, highlighting the overall facility layout and environment; its deployment within a rack-mounted system for high-bandwidth applications; a close-up of the chip’s circuitry and board connections; and a detailed view of the chip core. Together, these images highlight MTIA’s advanced design, connectivity, and its role in enhancing the performance and efficiency of AI tasks across Meta’s platforms.

synchronization. These features require kernel developers to adopt new pipeline structures that differ significantly from traditional warp-centric Ampere kernels.

This hardware diversity renders manual cross-platform kernel development economically infeasible, as each hardware platform requires complete reimplementations of thousands of distinct kernel variants with hardware-specific optimization strategies that do not transfer across architectural boundaries. Each platform-specific implementation demanding 2-8 weeks of expert optimization effort. The resulting implementation matrix— $O(\text{operators} \times \text{hardware platforms})$ —creates an unsustainable maintenance burden, further exacerbated by 12-18 month hardware generation update cycles that invalidate existing optimizations and necessitate complete kernel redesigns to exploit new architectural features.

Model Diversity Across Ranking Stages. As shown in Table 1, modern ads ranking employs multi-stage architectures where computational characteristics vary dramatically across pipeline stages. Retrieval models process millions of candidates using lightweight scoring functions optimized for both throughput and latency, employing approximate nearest neighbor search and efficient embedding operations. Early-stage ranking models apply moderate-complexity neural networks to thousands of candidates, balancing computational cost against filtering accuracy through pruned architectures and quantized operations. Second-stage ranking models execute heavyweight deep neural networks on hundreds of candidates with strict sub-100ms latency requirements, increasingly incorporating Transformer-based architectures that introduce distinct computational patterns. These Transformer-based ranking models [Zhai et al. 2024, Zeng et al. 2025, Zhang et al. 2024] introduce $10\text{-}100\times$ computational complexity increases per request compared to traditional dense architectures, requiring specialized attention kernels and sequence processing operations. Orthogonally, production recommendation models employ significantly larger embedding tables—often exceeding 100GB—that stress memory capacity and bandwidth, necessitating specialized embedding lookup and aggregation kernels optimized for irregular memory access patterns.

Each stage demands distinct kernel optimization strategies: retrieval favors batched operations maximizing memory bandwidth utilization; early-stage ranking requires balanced compute-memory workloads; final-stage ranking demands maximum single-request performance through aggressive operator fusion and specialization.

Ranking Stages	Description	Production Models	Total Models	Model Complexity
Early stage ranking	Rank 10K ads	≥ 150 models	≥ 500 models	0.01-0.1 GFLOPS/request
Late stage ranking [†]	Rank 100 ads	≥ 200 models	≥ 1000 models	0.2-2 GFLOPS/request

Table 1 Examples of some production models distribution at Meta. [†]Transformer-based sequence models require significantly higher computational complexity at ~ 80 GFLOPS/request, representing a 10-100 $\times$ increase over traditional dense ranking models.

Serving Paradigm 1: Overall Latency Ms				
Compute Tier	P50	P75	P90	P99
Client $\rightarrow$ MTIA Tier	39	44	46	61
Serving Paradigm 2: Overall Latency Ms				
Compute Tier	P50	P75	P90	P99
α : Client $\rightarrow$ CPU Tier	58	65	73	97
β : CPU Tier $\rightarrow$ MTIA Tier	42	48	51	57
γ : Data Preproc Execution	4	7	10	16
δ : Extra Network Latency	$\delta = \alpha - \beta - \gamma = 10 \sim 20ms$			

Table 2 Latency comparison of monolithic versus disaggregated serving paradigms for a production MTIA model. Paradigm 1 executes preprocessing client-side, achieving 61ms P99 latency with direct client $\rightarrow$ remote MTIA communication. Paradigm 2 introduces a dedicated CPU tier for preprocessing scalability, incurring additional network hops that increase P99 latency to 97ms. The extra network latency ($\delta = \alpha - \beta - \gamma \approx 10 \sim 20ms$) represents pure architectural overhead with no computational benefit, demonstrating the cost of disaggregated serving when preprocessing operators lack native accelerator implementations.

This architectural diversity—spanning from traditional MLPs to attention-based sequence models—prevents one-size-fits-all kernel optimization approaches and necessitates model-aware kernel generation strategies that can synthesize implementations for both established operators and rapidly emerging architectural patterns.

Kernel Diversity Beyond GEMM. While dense matrix multiplication (GEMM) operations benefit from mature, highly-optimized libraries (cuBLAS, FBGEMM, DeepGEMM) [PyTorch 2025a, DeepSeek 2025], production ads ranking workloads exhibit fundamentally different computational characteristics that necessitate broader kernel coverage. Ads ranking models execute over 200 distinct data preprocessing operators as integral components of model inference pipelines, where raw features undergo transformation before feeding subsequent neural network layers. These transformations [Zhao et al. 2022] span feature derivation (bucketization, set operations, n-gram hashing), dense normalization (variance-stabilizing transformations, one-hot encoding), and sparse normalization (cryptographic hashing with modulo reduction and type downcasting to map categorical features to embedding indices, top-k truncation with score-based ranking).

While individually exhibiting low arithmetic intensity compared to GEMM operations, kernel availability for these preprocessing operators fundamentally determines deployment architecture. The lack of optimized preprocessing kernels on AI accelerators creates a binary constraint: without native accelerator implementations, models become ineligible for unified accelerator deployment. Executing preprocessing on accelerator host CPUs proves infeasible: (1) host CPUs in accelerator servers are underprovisioned compared to dedicated CPU tiers, lacking capacity for preprocessing-intensive workloads; (2) heterogeneous preprocessing demands—varying compute and memory bandwidth requirements—contend with accelerator I/O and system management for limited host resources; (3) allocating additional host CPU capacity negates accelerator consolidation’s economic and power efficiency benefits.

This constraint forces disaggregated serving architectures partitioning models across compute tiers: preprocessing on CPU servers, neural network computation on accelerators (GPU, AMD, MTIA). While enabling independent tier scaling—a desirable microservice property [Audibert et al. 2023, Murray et al. 2021]—disaggregation **imposes severe penalties** for latency-critical models at multi-megawatt serving capacity. Table 2

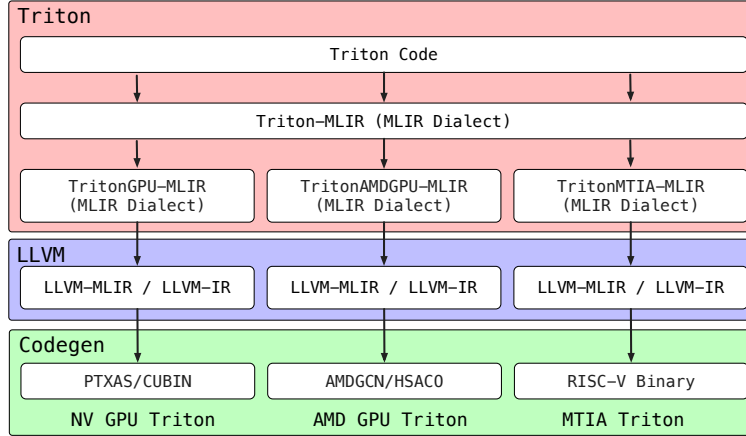


Figure 2 Triton multi-target compilation architecture. Source code transforms through progressive MLIR lowering stages—platform-independent Triton-MLIR, hardware-specific GPU/AMDGPU/MTIA dialects, LLVM-IR—generating native binaries for NVIDIA (PTX/CUBIN), AMD (AMDGCN/HSACO) [Zhang and Wang 2025], and MTIA (RISC-V) platforms.

quantifies these costs for a production MTIA deployment. Paradigm 1 executes preprocessing client-side before accelerator invocation (P99: 61ms). Paradigm 2 introduces a dedicated CPU preprocessing tier, incurring additional network hops increasing P99 to 97ms—a 25% degradation (compared to expected $57 + 16 \approx 73$ ms with zero network overhead). The 10-20ms network overhead represents pure architectural tax consuming a substantial portion of the sub-100ms latency budget required for ads serving. This fragmentation compounds through: (1) cross-node serialization and communication costs; (2) cascading failure modes across service tiers; (3) operational complexity from synchronized deployments and version compatibility; (4) increased TCO from redundant CPU infrastructure.

The architectural implications prove severe: preprocessing kernel absence creates a binary deployment constraint rather than incremental performance loss. A single missing preprocessing operator—regardless of arithmetic intensity—blocks entire model deployments on new accelerators, forcing disaggregated architectures with system-level costs exceeding individual kernel inefficiency. This constraint paradoxically elevates preprocessing operators to equal or greater priority than compute-intensive kernels: while suboptimal GEMM implementations degrade performance incrementally, missing preprocessing kernels impose architectural penalties preventing monolithic deployment entirely. Consequently, comprehensive kernel coverage enabling monolithic accelerator deployment—where preprocessing and neural network computation co-locate—emerges as a first-order requirement for cost-effective serving. This architectural imperative motivates KernelEvolve’s automated generation framework synthesizing optimized implementations across the complete operator-hardware matrix, treating data preprocessing as first-class optimization targets alongside traditional compute kernels.

Emerging AI-Powered Kernel Authoring. Recent advances in large language models have catalyzed a wave of AI-powered coding agent systems, with several targeting GPU kernel optimization. Academic efforts include KernelBench [Ouyang et al. 2025], which benchmarks LLM capabilities across three difficulty levels (operator, kernel fusion, full model) spanning canonical GPU operators, and AutoTriton [Li et al. 2025] from Tsinghua, which applies reinforcement learning to Triton programming. Industry initiatives span multiple organizations: Meta’s KernelLLM [Fisches et al. 2025] and CWM [Carbonneaux et al. 2025] explore code generation with world models; Amazon’s TritonRL [Woo et al. 2025] trains LLMs for Triton synthesis; AMD’s GEAK-agent [Wang et al. 2025b] targets Triton kernel generation with agentic workflows on AMD MI300X and MI250; Cognition AI’s Kevin [Baronio et al. 2025] employs multi-turn RL for CUDA kernel generation. These systems demonstrate that AI agents can generate competitive implementations on isolated OSS benchmarks through iterative refinement and learned optimization strategies.

However, these remain research prototypes with fundamental limitations for production deployment. (1) narrow optimization scope: systems target isolated subproblems—AutoTriton focuses on RL post-training for Triton, Kevin optimizes CUDA generation—without addressing end-to-end kernel lifecycle management from

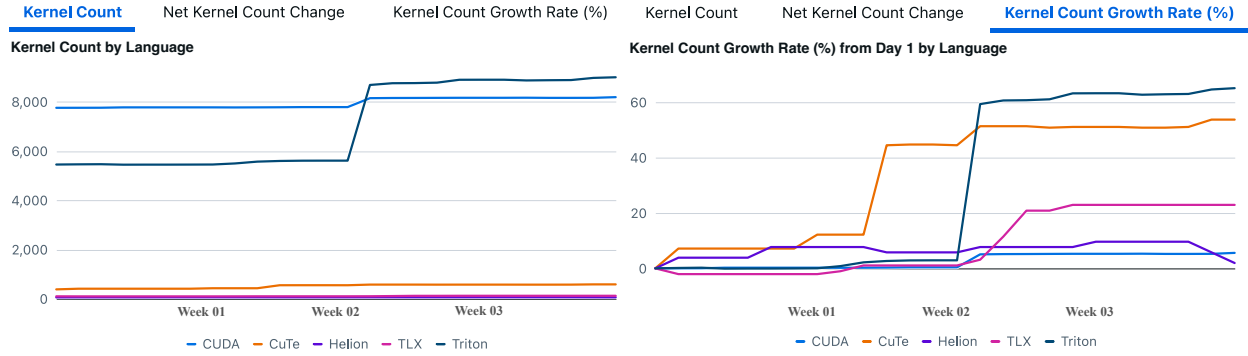


Figure 3 Triton overtakes CUDA as dominant kernel programming model at Meta. Left: Triton has grown to over 8,000 kernels, surpassing CUDA’s stagnant legacy codebase, while emerging DSLs (CuTe, TLX, Helion) remain under 600. Right: Growth trajectories show Triton’s 60% expansion rate driving this transition, with CuTe at 50% following November deployment. This shift toward higher-level DSLs—while maintaining legacy CUDA and introducing new abstraction (TLX)—creates programming model fragmentation across 5+ languages, motivating KernelEvolve’s automated synthesis approach.

synthesis to deployment. (2) synthetic evaluation: benchmarks use canonical operators with static tensor shapes rather than production workloads exhibiting dynamic batching, variable sequence lengths, and domain-specific transformations (e.g., jagged tensor operations, cryptographic hashing for feature engineering). (3) single-platform focus: most target homogeneous NVIDIA environments without cross-platform synthesis for heterogeneous accelerator fleets (NVIDIA, AMD, custom ASICs). (4) limited agent capabilities: existing systems lack fully autonomous workflows encompassing automated synthesis, multi-level correctness verification (unit tests, integration tests, numerical accuracy), hierarchical profiling feedback (system, kernel and intra-kernel granularities), and persistent knowledge bases via filesystem that enable context-aware prompt synthesis by dynamically retrieving relevant optimization patterns, hardware specifications, and historical profiling data. (5) absence of inference-time scaling: no system employs large-scale search strategies (greedy, Monte Carlo Tree Search, evolutionary algorithms) that iterate hundreds to thousands of optimization steps per kernel, a capability critical for achieving expert-level performance. (6) no checkpointing support: systems restart from scratch on failure rather than resuming from intermediate states, making multi-hour optimization runs brittle and resource-inefficient. These gaps—spanning workload realism, hardware heterogeneity, agent autonomy, search scalability, and operational robustness—prevent existing prototypes from meeting production requirements.

KernelEvolve. To address these fundamental limitations, we propose KernelEvolve, an agent-based framework that automates the generation and optimization of high-performance compute kernels for ads ranking model serving across heterogeneous hardware architectures. Responding to the programming model evolution in Figure 3—where Triton has emerged as the dominant DSL with 60% growth—KernelEvolve adopts Triton as its primary target, capitalizing on its cross-platform support across NVIDIA, AMD, and MTIA accelerators (see Figure 2). We additionally target Triton-TLX, enabling low-level hardware-specific tuning while maintaining Triton’s portability advantages.

Figure 4 demonstrates KernelEvolve’s effectiveness across diverse production workloads and hardware platforms. KernelEvolve achieves substantial speedups spanning LLM inference workloads (Llama-3.1-8B: Vanilla Attention 4.6 $\times$, SDPA-MLP 3.3 $\times$), convolutional transformers (conv1d: 6.5 $\times$, conv2d: 4.7 $\times$), memory-bound data preprocessing operators critical for model enablement (MapId: 4.1 $\times$, MBDT: 9.3 $\times$, Batch Event Truncate: 9.8 $\times$), compute-intensive fusion kernels in ranking models (WuKong Optimized FM: 4.0 $\times$, InterFormer PFFN: 2.5 $\times$), MTIA-specific optimizations (RMSNORM 2D backward: 17 $\times$), and retrieval operations (Sparse Inverted Index: 1.25 $\times$). These results validate KernelEvolve’s ability to generate production-grade implementations across the full spectrum of recommendation and LLM inference workloads—from compute-bound fusion to memory-bound preprocessing—while addressing hardware-specific optimization challenges on proprietary accelerators. Detailed evaluation and analysis are presented in Section 5.

KernelEvolve leverages agentic AI capabilities to establish an end-to-end kernel generation service that

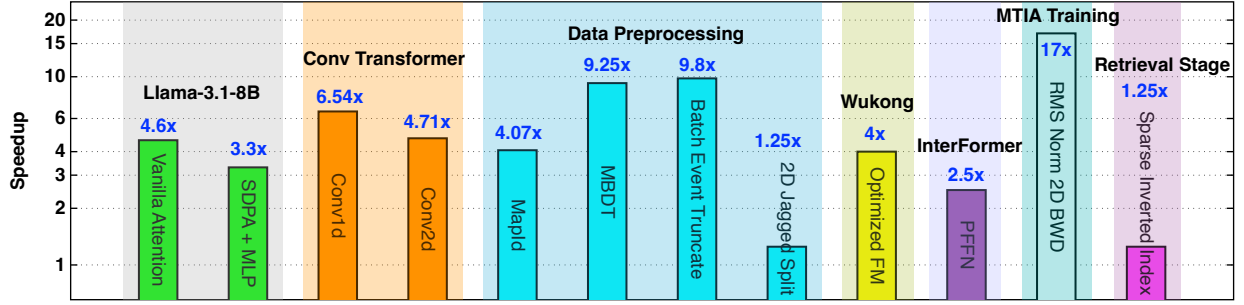


Figure 4 KernelEvolve achieves 1.25-17 $\times$ speedups across Meta LLMs and production use cases, spanning convolutional Transformers, data preprocessing operators, and recommendation systems, over heterogeneous AI hardware.

autonomously handles kernel synthesis, compilation, profiling, correctness verification, and performance benchmarking. This approach fundamentally transforms traditional kernel development from a manual, expertise-dependent process to an automated, scalable service maintaining production-grade performance standards while adapting to evolving model architectures and hardware diversity.

This paper makes the following contributions:

- We present KernelEvolve, the first production-grade AI-powered kernel optimization system deployed at industrial scale for recommendation model training and inference. KernelEvolve operates continuously in Meta’s production infrastructure, autonomously generating optimized Triton kernels for hundreds of models serving billions of users daily. Our agent-based approach achieves production-grade reliability while exploring optimization strategies infeasible through manual development, and demonstrates how structured knowledge bases encoding hardware-specific constraints enable effective kernel generation for proprietary architectures absent from LLM training corpora.
- We demonstrate KernelEvolve’s autonomous kernel optimization achieving competitive performance with expert manual implementations while reducing development time from weeks to hours. Through diverse production use cases spanning ads training and serving workloads, KernelEvolve-generated kernels achieve 1.2 to 17 times speedup over the PyTorch baselines, demonstrating that automated synthesis can exceed state-of-the-art compiler-generated code for domain-specific operators. We analyze optimization trajectories across diverse operator types and hardware platforms, identifying patterns in successful kernel generation strategies that inform future automated optimization systems.
- We share insights from operating KernelEvolve in production environments, including failure mode analysis, debugging strategies for incorrect kernel generation, performance validation methodologies ensuring production safety, and organizational integration patterns for adopting automated kernel generation workflows.

2 Background

2.1 Meta’s Recommendation Model

Meta’s products—Facebook, Instagram, Reels, Threads—rely on deep learning recommendation models (DLRM) [Naumov et al. 2019] delivering personalized content including advertisements, short videos, and posts. Llama-based generative AI additionally powers advertising features such as image and text generation [Dubey et al. 2024]. Recommendation infrastructure processes trillions of daily inferences through multi-stage pipelines exhibiting substantial architectural diversity driven by stage-specific computational and latency constraints.

Multi-Stage Ranking Pipeline. The recommendation funnel comprises three stages with distinct characteristics: Retrieval processes millions of candidates narrowing to 10K-100K items through low-complexity models operating at large batch sizes with significant preprocessing overhead; Early-stage ranking refines thousands of

candidates to hundreds using moderate-complexity models; Late-stage ranking scores hundreds of candidates with heavyweight models (up to 2 GFLOPS per sample) incorporating high-order feature interactions through hierarchical architectures (DHEN [Zhang et al. 2022], Wukong [Zhang et al. 2024]). Production late-stage models exhibit over $60\times$ complexity variation reflecting diverse quality-latency trade-offs.

DLRM Architecture. Traditional recommendation models combine sparse feature embeddings (categorical inputs: post IDs), dense feature transformations via MLP (continuous values: age, time), and interaction layers modeling cross-feature dependencies. Recent architectures incorporate Transformer-like components for sequence modeling—HSTU [Zhai et al. 2024] processes user history through jagged attention mechanisms, while InterFormer [Zeng et al. 2025] enables bidirectional information flow between sequential and non-sequential features through personalized attention—introducing $10\text{--}100\times$ per-request complexity increases versus traditional DLRM while demanding larger embedding tables stressing memory bandwidth.

Emerging Generative Recommendation. Recent recommendation systems explore generative approaches treating recommendation as sequence generation. Architectures like OneRec [Zhou et al. 2025a,b] employ quantization techniques (RQVAE [Rajput et al. 2023], RQ-Kmean [Luo et al. 2025]) converting continuous embeddings into discrete semantic IDs for LLM-based modeling. These introduce computational patterns—autoregressive decoding, token-based retrieval, quantization operations—requiring kernel support beyond traditional DLRM operators.

This architectural diversity creates heterogeneous kernel optimization requirements. Retrieval prioritizes throughput at large batch sizes; late-stage ranking demands compute-intensive fused interactions under sub-100ms latency; sequence models require jagged tensor operations for variable-length histories; generative recommendation introduces quantization and autoregressive decoding primitives. Each stage combines these specialized operators with data preprocessing transformations, creating a diverse operator landscape requiring hundreds of optimized kernel implementations.

2.2 Data Preprocessing Operations

Data preprocessing transforms raw features into model-ready inputs, executing transformations on batched data before feeding neural network stages. During training, preprocessing executes on distributed workers [Zhao et al. 2022]; during inference, preprocessing operators are embedded within model modules as integral components—executing synchronously with model invocation under strict latency constraints where preprocessing latency directly impacts end-to-end inference response time.

Feature Types and Memory Layout. Ads ranking models consume four primary feature types with distinct memory representations [Liao et al. 2025]: (1) Dense features (`float32`) encode continuous user/item attributes (age, click-through rates, engagement scores); (2) Sparse features (`list<int64>`) represent categorical IDs (page IDs, post IDs) as variable-length lists requiring jagged tensor support; (3) Weighted sparse features (`list<pair<int64, float32>>`) associate scores with IDs for ranking-based operations; (4) Event-based features capture user history events as temporal sequences, increasingly adopted in production models for improved prediction quality through sequential interaction modeling.

Preprocessing Transformation Categories. Production preprocessing pipelines execute three operation classes: (1) Dense normalization applies statistical transformations (BoxCox, Logit) and one-hot encoding to grouped features, followed by linear scaling (shift, multiplication) across the feature matrix; (2) Sparse processing truncates ID lists via top-k selection (optionally sorted), applies cryptographic hashing mapping IDs to embedding table indices, and downcasts hashed values from `int64` to `int32` for memory efficiency; (3) Feature derivation computes new features from existing ones through complex operator compositions—bucketizing continuous values into categorical bins, set operations across multiple sparse lists, n-gram hashing for text features. Feature derivation comprises hundreds of operator branches in abstract syntax trees executing over feature subsets, creating substantial kernel diversity beyond standard neural network primitives.

This preprocessing complexity—hundreds of operators with irregular memory access patterns, data-dependent control flow, and sparse computation characteristics—fundamentally determines serving paradigm in production systems, as discussed in Section 1. The diversity and computational significance of preprocessing operators motivate comprehensive kernel coverage as a first-order architectural requirement rather than focusing solely on compute-intensive operations like GEMM.

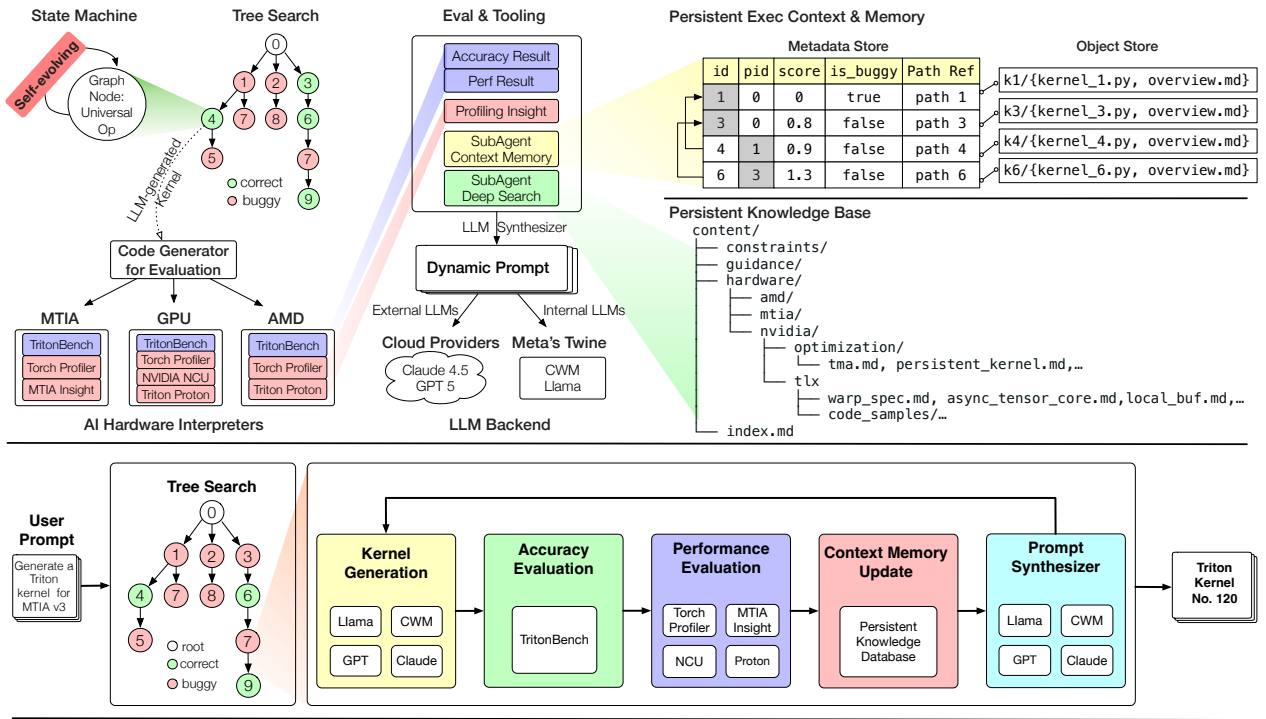


Figure 5 KernelEvolve System Architecture (top) and Execution Workflow (down). KernelEvolve employs a self-improving state machine with tree search to explore and validate kernel optimizations. The system integrates evaluation tooling (accuracy, performance, profiling), specialized sub-agents for context management and deep search, and AI hardware interpreters for MTIA, GPU, and AMD platforms. An LLM synthesizer generates dynamic prompts, which are then used by external (Claude 4.5, GPT-5) or internal (Meta’s CWM) LLM backends to generate Triton kernel candidates. Persistent context includes a metadata store tracking execution scores and parent-child relationships in the search tree (connected to the object store via path references), an object store for kernel files, and a knowledge base that serves as a retrieval system for hardware constraints and optimization guidance to support LLM context augmentation.

3 System Architecture

Figure 5 illustrates the KernelEvolve system architecture and its optimization workflow. The system centers around a self-improving state machine that drives exploration of the kernel optimization space through tree search strategies. Recent advancements [Jiang et al. 2025] demonstrate state-of-the-art performance by conceptualizing iterative experimentation as tree search over potential solutions. Building on this insight, KernelEvolve employs multiple search strategies including greedy search for rapid initial solutions, Monte Carlo Tree Search (MCTS) [Kocsis and Szepesvári 2006] for balanced exploration-exploitation, and evolutionary algorithms [De Jong 2017] for population-based optimization. Each iteration begins with the LLM synthesizer generating dynamic prompts augmented with context from specialized sub-agents. The context memory sub-agent maintains relevant historical information from previous iterations and analyzes the current state of the search tree to guide exploration strategy, while the deep search sub-agent further enriches these prompts through retrieval from the persistent knowledge base, which organizes hardware-specific constraints for different accelerators (AMD, MTIA, NVIDIA), optimization guidelines, and code samples in a hierarchical file system structure. The augmented prompts are then processed by either external LLM backends (Claude 4.5, GPT-5) or internal models (Meta’s CWM [Carbonneaux et al. 2025] and Llama [Dubey et al. 2024] on Twine [Tang et al. 2020]) to generate Triton kernel candidates.

Generated kernels undergo comprehensive evaluation across multiple dimensions. TritonBench validates correctness by comparing generated Triton kernels against PyTorch reference implementations while measuring speedup over baselines. System-level profiling via Torch Profiler captures CPU/GPU time, kernel launch overhead, and function execution durations to identify host-device bottlenecks. However, modern AI accelerator optimization faces a fundamental challenge: performance signals are fragmented across abstraction layers—DSLs, compiler IR, CUDA/PTX/SASS, runtime APIs, and hardware counters—requiring manual correlation across siloed tools. Intra-kernel tracers such as Triton Proton [Zhou et al. 2025c] expose instruction-level behavior, kernel profilers report occupancy and memory throughput, while system profilers capture execution timelines and communication patterns. No single tool provides complete stack visibility. To address this fragmentation, Meta developed MPP (Multi-Pass Profiler), a federated tooling framework that unifies instrumentation, compiler transforms, profiling, and trace synthesis as composable job graph tasks. Integrated with existing ecosystems including MLIR [Lattner et al. 2020], Proton [Zhou et al. 2025c, Guan et al. 2025], NCU [Nvidia 2025], and NVBit [Villa et al. 2019], MPP enables KernelEvolve to automate cross-stack experiments and access metrics previously gated by tool fragmentation. Hardware-specific profilers (MTIA Insight and Triton Proton) provide platform-specific insights across MTIA, GPU, and AMD interpreters. The evaluation framework in KernelEvolve produces accuracy validation, performance metrics, and profiling insights that collectively determine each search tree node’s status (correct or buggy) and assign optimization insights to guide automatic exploration strategy.

The persistent storage layer enables continuous learning and knowledge accumulation across optimization runs. The metadata store maintains a comprehensive execution context for each kernel candidate, including unique identifiers, parent-child relationships that encode the search tree structure, quality scores from evaluation results, and boolean flags (`is_buggy`) indicating whether the kernel has execution errors or accuracy failures. Path references in the metadata store link to the object store, which maintains the actual kernel implementations organized by unique identifiers along with associated Triton kernel files and overview documentation (`overview.md`) containing profiling results analysis and optimization recommendations. This separation of metadata and content allows efficient querying and comparison of kernel variants while preserving complete implementation history. The architecture’s modularity supports flexible deployment configurations with different LLM backends and hardware targets while maintaining a consistent optimization methodology. By systematically exploring the solution space and learning from both successful optimizations and failed attempts, KernelEvolve progressively generates higher-quality kernel implementations that approach theoretical performance limits for given hardware constraints.

The following sections detail the design and implementation of key system components.

3.1 Tree Search and State Machine

Given a kernel generation problem, KernelEvolve maintains a search graph $G_t = (V_t, E_t)$ that evolves over multiple iterations $t = 0, 1, \dots$, where each node $v \in V_t \subseteq \mathcal{S}$ represents a kernel implementation artifact belonging to the set of all possible artifacts $\mathcal{S}$, and each directed edge $(v_i, v_j) \in E_t$ represents a transformation from kernel v_i to v_j . The root node v_0 represents the initial specification or baseline implementation. At each iteration, the agent executes three fundamental operations: (1) **selects** promising nodes via a selection policy π_{sel} , (2) **applies** a transformation operator to generate new kernel candidates, and (3) **scores** the resulting solutions via a fitness function $\mathcal{F}$ that evaluates correctness and performance.

Recent research demonstrates that LLMs alone are insufficient for effectively solving open-ended optimization tasks [Nathani et al. 2025]. Performance significantly improves when augmented with external tools [Qin et al. 2024, Schick et al. 2023], execution feedback [Gehring et al. 2024], and solutions addressing context limitations. Developing high-performance kernels requires iterative experimentation where insights from prior attempts inform subsequent refinements. Building on recent advancements [Jiang et al. 2025, Toledo et al. 2025b], which achieve state-of-the-art results by conceptualizing iterative experimentation as tree search over potential solutions, we formalize kernel optimization as a graph-based search algorithm that systematically explores the solution space.

Graph-Based Search Framework. We model KernelEvolve as a graph-based search algorithm specified by the tuple $(\mathcal{F}, \pi_{\text{sel}}, \mathcal{O}, \tau)$, where:

- **Fitness Function** $\mathcal{F} : \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ estimates the quality of a kernel implementation node $v \in V_t$ through the speedup achieved by the generated Triton kernel relative to the PyTorch compiled reference code [Ansel et al. 2024]. Specifically, for a generated Triton kernel with execution time t_{triton} and PyTorch compiled baseline with execution time t_{pytorch} , the fitness is computed as $\mathcal{F}(v) = \frac{t_{\text{pytorch}}}{t_{\text{triton}}}$. Kernels that fail correctness validation (numerical accuracy checks against reference code) or encounter compilation/runtime errors are assigned $\mathcal{F}(v) = 0$. This fitness measure directly captures the performance optimization objective while ensuring correctness as a hard constraint.
- **Selection Policy** $\pi_{\text{sel}} : 2^{V_t} \rightarrow 2^{V_t}$ chooses a subset of nodes $U_t \subseteq V_t$ for expansion, guided by a heuristic function $h : V_t \rightarrow \mathbb{R}$ that assigns scalar estimates to each node. Different instantiations support various search strategies: greedy search selects the single highest-scoring node, Monte Carlo Tree Search (MCTS) balances exploration-exploitation via upper confidence bounds for trees (UCT), and evolutionary algorithms maintain populations of diverse candidates for crossover and mutation operations.
- **Universal Operator** $\mathcal{O} : \mathcal{S} \times \mathcal{C} \rightarrow \mathcal{S}$ is a single transformation function that generates new kernel candidates from existing implementations, where $\mathcal{C}$ represents the contextual information (profiling results, error messages, hardware constraints, historical optimizations) that guides the transformation. Unlike traditional approaches that employ multiple specialized operators (Draft, Debug, Improve) with fixed prompting strategies, KernelEvolve’s universal operator dynamically adapts its behavior based on runtime context through retrieval-augmented prompt synthesis.
- **Termination Rule** τ halts search when computational budgets (wall-clock time or maximum number of artifacts) are exhausted, progress stalls, or fitness thresholds are achieved.

The Operator Bottleneck and Universal Operator Design. Prior research indicates that performance bottlenecks in LLM-based code generation stem primarily from operator design rather than search algorithms [Toledo et al. 2025b]. Traditional multi-operator frameworks face a fundamental limitation: each operator (e.g., Debug, Improve) is associated with a static prompt template that cannot adapt to runtime execution context. For instance, Debug operators employ fixed error-focused prompts regardless of whether the underlying issue stems from algorithmic errors, memory access patterns, or hardware-specific constraints, while Improve operators use performance-focused prompts that remain unchanged despite varying bottleneck characteristics (compute-bound vs. memory-bound vs. synchronization overhead). This static prompting strategy imposes cognitive constraints on the model’s reasoning process, potentially misleading the model by framing optimization problems through predefined lenses that may not align with the actual runtime context. The operator-specific prompting creates artificial boundaries in the solution space, preventing the model from simultaneously reasoning about correctness, performance, and architectural trade-offs based on the specific execution characteristics observed at runtime.

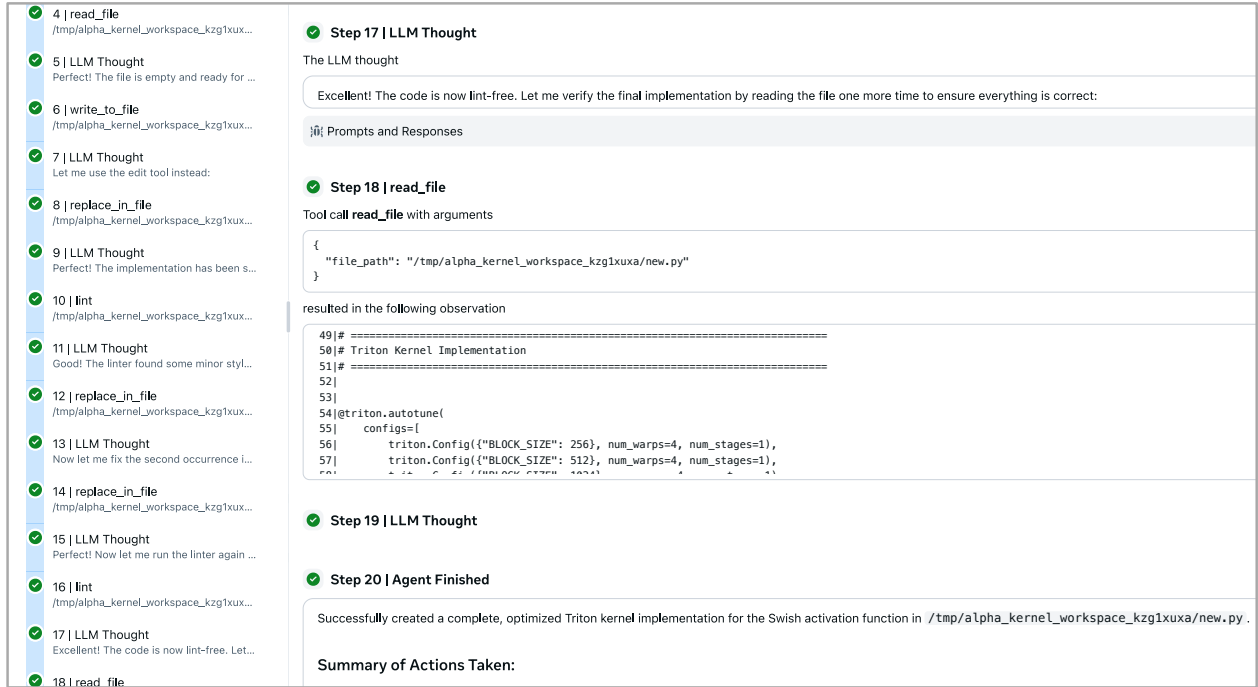


Figure 6 Universal operator’s agentic kernel generation workflow for Swish activation. Agent iteratively reads, analyzes, modifies, and validates code through tool invocations (`read_file`, `write_to_file`, `replace_in_file`, `lint`), guided by LLM reasoning at each step. The workflow completes after 20 steps, producing lint-free optimized Triton kernel with autotune configurations.

To address this limitation, KernelEvolve employs a **single universal operator** that dynamically adapts its behavior based on runtime context rather than predefined operator categories (Figure 6). Instead of fixed prompt templates associated with specific operators, we introduce a retrieval-augmented dynamic prompting mechanism (detailed in Section 3.2) that synthesizes contextually optimal prompts at each iteration. This design choice allows the LLM to reason holistically about kernel optimization without artificial constraints imposed by operator boundaries. By unifying operator functionality under a single context-aware interface, KernelEvolve enables more flexible exploration strategies where each generation step can simultaneously address multiple optimization objectives—correcting numerical errors, improving memory access patterns, exploiting hardware-specific features, and refining algorithmic approaches.

3.2 Agentic Retrieval & Self-Managed Context

KernelEvolve employs a retrieval-augmented approach where contextual information is dynamically loaded into the LLM’s context at runtime rather than maintaining complete historical knowledge in working memory. This design philosophy aligns with recent agentic coding systems such as Anthropic’s Claude Code [Anthropic 2025], which demonstrates that complex analysis over large datasets can be performed through targeted queries and incremental loading rather than exhaustive context consumption. This approach reflects human cognitive patterns: rather than memorizing entire corpuses of information, humans introduce external systems—file hierarchies, databases, indexing structures—to retrieve relevant information on demand. For KernelEvolve, this retrieval-based dynamic prompting eliminates cognitive constraints imposed by static prompt templates, enabling the LLM to reason freely about optimization strategies guided by actual runtime execution characteristics rather than predefined operator semantics.

KernelEvolve operationalizes this architecture through a two-stage pipeline implemented by specialized sub-agents. The **context memory sub-agent** analyzes dynamic runtime artifacts—kernel implementations, profiling measurements, error diagnostics, correctness validation results—to diagnose performance bottlenecks and synthesize optimization directives. These analysis outputs parameterize the **deep search sub-agent**, which performs targeted retrieval from a persistent knowledge base containing hardware constraints, optimization

patterns, and debugging methodologies. This staged design reflects a fundamental principle: effective knowledge retrieval requires runtime context to determine retrieval targets. The context memory sub-agent identifies which optimization challenges require attention; the deep search sub-agent retrieves domain knowledge addressing those specific challenges. By conditioning knowledge base retrieval on runtime analysis rather than static heuristics, KernelEvolve ensures retrieved content directly targets observed bottlenecks, maintaining context window efficiency while avoiding irrelevant generic guidance.

3.2.1 Deep Search Sub-Agent

The persistent knowledge base implements a hierarchical file system encoding domain expertise across hardware platforms, optimization strategies, debugging patterns, and language constraints. This organization exploits structural metadata—folder hierarchies, naming conventions, and file relationships—as retrieval signals that guide agentic search without explicit semantic annotation. A comprehensive index (`index.md`, Figure 5) maintains the complete directory tree with inline module documentation, serving dual purposes: human-readable reference and machine-parseable navigation structure for automated retrieval.

Hierarchical Taxonomy. The knowledge base partitions content across three primary categories addressing distinct optimization concerns:

- **Constraints** (`constraints/`): Enforces correctness requirements through anti-cheating rules (prohibiting cross-platform abstractions, external library dependencies), forbidden patterns (direct CUDA API usage, incomplete test coverage), and output format specifications. These constraints ensure generated kernels represent authentic Triton implementations rather than wrappers around pre-compiled libraries or high-level PyTorch operations that bypass kernel-level optimization.
- **Guidance** (`guidance/`): Provides platform-agnostic optimization knowledge organized by concern: debugging methodologies (error interpretation, numerical stability), performance tuning (autotuning, block sizing, fusion strategies), and Triton language idioms (data types, indexing patterns, memory primitives). This cross-platform content transfers across hardware architectures, establishing foundational implementation patterns before platform-specific specialization.
- **Hardware** (`hardware/`): Encodes platform-specific knowledge for NVIDIA GPUs, AMD GPUs, and MTIA accelerators. Each platform subtree maintains architectural documentation (compute hierarchies, memory subsystems, execution models), platform-specific debugging guidance (common pitfalls, precision issues), and advanced optimization techniques. NVIDIA content includes Hopper-generation features (Tensor Memory Accelerator, warp specialization via Triton Low-level Extensions (TLX)), persistent kernel patterns, and generation-specific optimizations. Hardware modules comprise 15-40 documents per platform—totaling ≥ 100 documents—reflecting the depth of architectural specialization required for production-grade kernel optimization.

Index-Guided Retrieval. The `index.md` file implements structured navigation enabling efficient content discovery. Upon receiving runtime feedback (profiling metrics, error diagnostics), the deep search sub-agent executes two-stage retrieval: (1) queries the index to identify relevant modules based on hardware platform, bottleneck type, and optimization phase; (2) fetches targeted content from identified modules. Memory bandwidth bottlenecks on NVIDIA H100 trigger index queries returning `hardware/nvidia/optimization/{tma, shared_memory, on_device_tma}.md`. The hierarchical structure enables efficient pruning: top-level platform selection (`hardware/{nvidia|amd|mtia}/`) eliminates irrelevant architectures; folder hierarchies encode concern taxonomies (`arch/`, `debug/`, `optimization/`); file naming conventions signal specificity (`memory_hierarchy.md` versus `on_device_tma.md`).

Progressive Specialization. Content organization supports iterative refinement throughout optimization trajectories. Initial generation retrieves broad guidance—Triton basics, general optimization principles, correctness requirements. Subsequent iterations navigate progressively specialized content guided by profiling feedback. A representative trajectory for compute-intensive GEMM on H100: (1) `hardware/nvidia/arch/tensor_cores.md` establishing Tensor Core capabilities; (2) `hardware/nvidia/tlx/{overview, warp_specialization, async_tensor_core_operations}.md` introducing fine-grained control through producer-consumer warp patterns and asynchronous matrix operations; (3) `code_samples/{hopper-gemm-pipelined, hopper-gemm-ws}.py` providing complete reference implementa-

tions. This progression from high-level tensor core usage to expert-level TLX pipelined and warp specialization exemplifies support for both novice implementations and production-grade optimizations approaching theoretical hardware limits.

3.2.2 Context Memory Sub-Agent

The context memory sub-agent bridges the persistent knowledge base and runtime optimization state through a two-tier storage architecture separating metadata from content. As shown in Figure 5, each explored search graph node persists to a metadata store (relational database) containing: unique identifiers (`id`), parent references (`pid`) encoding tree structure, fitness scores (`score`), correctness flags (`is_buggy`), and path references (`path_ref`) linking to an object store containing kernel implementations (`kernel_n.py`), profiling results, and LLM-generated analysis reports (`overview.md`). This separation enables efficient metadata queries without loading large source files or profiling traces.

The relational storage architecture provides two critical capabilities beyond simple persistence. (1) **Distributed Concurrent Exploration.** It enables distributed concurrent exploration: multiple agents can simultaneously expand different nodes in the search graph, with the database providing transaction isolation and consistency guarantees. When KernelEvolve scales to dozens or hundreds of concurrent agents exploring thousands of optimization steps, maintaining an in-memory graph representation becomes infeasible. The metadata store allows agents to operate independently, querying only relevant subgraphs on demand. (2) **Complex Contextual Queries.** It supports complex contextual queries through SQL operations that logically reconstruct graph views without materializing the entire structure in memory [Liao and Abadi 2023]. The relational schema enables efficient graph traversal via recursive Common Table Expressions (CTEs). These queries enable sophisticated context construction: analyzing sibling node outcomes, retrieving strategies from high-performing ancestors, identifying global best solutions for comparison. (3) **Cross-Session Knowledge Reuse.** Persistence enables querying historically explored similar operators (matched by operator type, input shapes, hardware platform). When high-quality solutions exist, KernelEvolve initializes search with these implementations rather than generating from scratch [Liao et al. 2026]. Consider a new GEMM variant for transformer attention on AMD MI350: metadata queries identify 15 historical GEMM kernels, three achieving $> 1.5\times$ speedup through TLX warp specialization. KernelEvolve retrieves the highest-performing implementation (score 1.5) with its optimization report documenting successful strategies (async tensor core operations, double-buffered shared memory), then focuses exploration on problem-specific adaptations (different dimensions, fusion opportunities) rather than rediscovering fundamental patterns. This approach provides: (reduced time-to-solution starting from proven implementations, inference cost reduction eliminating redundant token consumption, and environmental impact reduction through decreased computational overhead. (4) **Fault Tolerance and Checkpointing.** Persistent storage enables fault-tolerant search with automatic checkpointing. When search processes crash or are interrupted, KernelEvolve reconstructs the complete search state from metadata: loading explored nodes, their parent-child relationships, fitness scores, and associated artifacts. Tree search resumes from the last successful iteration rather than restarting from scratch. This proves critical for long-running optimization campaigns where generating production-grade kernels may require hundreds of iterations spanning hours: hardware failures, deployment updates, or resource preemption no longer discard accumulated exploration progress. The metadata store serves as continuous checkpoint, with each node insertion atomically persisting exploration state.

Runtime Artifact Analysis. At each search node, KernelEvolve generates execution artifacts: kernel source, execution logs, correctness validation results, performance measurements (execution time, speedup), and profiling metrics (instruction latency, memory throughput, occupancy, synchronization overhead). The context memory sub-agent invokes the LLM to analyze these artifacts, producing structured reports diagnosing bottlenecks and recommending optimization strategies. Profiling revealing 30% occupancy on H100 with high shared memory pressure triggers analysis identifying register spilling and bank conflicts as root causes, recommending register usage reduction through value recomputation and warp-level memory access optimizations.

Dynamic Prompt Synthesis. The context memory sub-agent composes prompts for the universal operator combining: (1) current kernel implementation and execution history, (2) LLM-generated analysis reports, (3) retrieved knowledge base content, (4) hardware-specific constraints. This implements self-managed context windows maintaining only task-relevant information within token budgets (64K-1M tokens depending on LLM backend). When multiple optimization opportunities exist, the sub-agent prioritizes based on profiling

evidence, loading content for dominant bottlenecks while deferring secondary optimizations to subsequent iterations.

Iterative Refinement. The context memory maintains summaries of previous optimization attempts, enabling learning from successes and failures. If increasing block size fails to improve speedup or introduces correctness violations, subsequent iterations avoid that strategy and explore alternatives. This mirrors human debugging workflows where engineers track attempted fixes, analyze failures, and systematically explore solution spaces while avoiding dead ends. Combined with targeted knowledge retrieval, this enables efficient navigation of complex optimization landscapes compared to static prompts or naive trial-and-error approaches.

The persistent storage architecture scales efficiently: metadata queries execute in milliseconds even with millions of nodes, while object store access occurs selectively. Production deployments maintain search histories spanning months across hundreds of operator types and multiple platforms, creating continuously growing kernel expertise corpora benefiting all users while reducing aggregate inference costs.

3.2.3 MTIA Knowledge Injection

MTIA presents unique challenges for LLM-based kernel generation: unlike widely-documented GPU architectures (NVIDIA CUDA, AMD ROCm), MTIA's proprietary architecture and programming model remain largely absent from public training corpora. Pretrained LLMs lack knowledge of MTIA-specific hardware features, extended Triton language constructs, and optimization patterns. KernelEvolve addresses this knowledge gap through systematic injection of MTIA domain expertise into the persistent knowledge base, enabling the deep search sub-agent to retrieve MTIA-specific content that educates the LLM during kernel generation.

MTIA Triton Extensions. While Triton originated as a GPU-focused language, Triton-MTIA extends the base language to expose hardware-specific features fundamentally different from GPU programming models. The knowledge base documents three categories of extensions addressing distinct requirements:

- **Hardware Feature Exposure:** As shown in Figure 7, MTIA v2i architectures [Firoozshahian et al. 2023, Coburn et al. 2025] expose unique capabilities including Specialized Function Units (SFU) with lookup table (LUT) operations, inter-Processing Element (PE) communication primitives, and dual-core synchronization mechanisms. The knowledge base provides detailed documentation of libdevice APIs and hardware-specific compiler directives.

Libdevice APIs map Triton operations to hardware primitives: `tl.extra.libdevice.gelu(x)` compiles to SFU LUT queries rather than mathematical approximations, providing higher performance at potential accuracy cost. Documented operations include `exp`, `gelu`, `log`, `sigmoid`, `tanh`—each mapping to dedicated SFU instructions unavailable on GPU targets.

MTIA exposes hardware-specific performance tuning through compilation options during kernel invocation. Two critical options documented in the knowledge base enable pipeline parallelism optimization: (1) `cb_multiplier` (integer) increases Circular Buffer allocation by specified factors, allowing multiple operations to execute concurrently by expanding on-chip memory capacity; (2) `use_dual_core` (boolean) instructs the compiler to distribute operations between core A and core B core—executing DMAs on core A while core B performs vector instructions—improving throughput through heterogeneous execution. These options can be explored statically via `@triton.autotune` decorators that evaluate configuration combinations (e.g., `BLOCK_SIZE ∈ {32, 1024}`, `cb_multiplier ∈ {1, 8}`), with autotuning keyed to input dimensions (`key=["N"]`) to rerun when problem characteristics change.

- **Compute Helper Functions:** MTIA provides three categories of optimized compute helpers documented in the knowledge base: (a) `unary_elemwise_compute(op, x)` supporting 30+ operations including mathematical functions (`exp`, `log`, `sqrt`), activations (`relu`, `gelu`, `sigmoid`), and logical operations; (b) `binary_elemwise_compute(op, x, y)` for tensor-tensor operations including arithmetic, comparisons, and ML-specific functions (`gelu_backward_tanh`, `log_sigmoid_backward`); (c) `binary_elemwise_const_compute(op, x, const)` for tensor-scalar operations. These helpers compile to optimized vector instructions, providing higher performance than manual Triton implementations expressing equivalent semantics.
- **Custom Type System:** MTIA kernels operate on device-specific data structures including `TensorView` (tensor metadata with shape, stride, addressing information), `CoreID` (PE identification and chip topology),



Figure 7 MTIA 2i architecture featuring an 8×8 processing element (PE) array interconnected via network-on-chip. Each PE contains dual RISC-V cores and specialized fixed-function units: Memory Layout Unit (MLU) for data transformation, Dot Product Engine (DPE) for matrix operations, Reduction Engine (RE) for aggregations, SIMD Engine (SE) for vector operations, and Command Processor (CP) for control flow (more details can be found in Section 3 of [Coburn et al. 2025]).

and `ExecutionGrid` (kernel launch configuration). The knowledge base documents type definitions via `@core.struct_type` decorators and associated utility functions, enabling kernels to directly manipulate MTIA runtime structures rather than relying on compiler-managed abstractions.

Advanced Synchronization and Communication Primitives. MTIA’s multi-PE architecture requires explicit control over inter-PE data movement and synchronization—capabilities absent from standard Triton. The knowledge base documents four critical extensions:

- **Cross-PE Broadcasting** (`direction` attribute in `tl.load`): Enables streaming memory between neighboring PEs, allowing multiple PEs reading identical memory to receive data from immediate neighbors rather than independent memory accesses. The `direction` parameter ("down" or "right") specifies propagation direction through the PE grid. Complementary `tl.consume()` operator reads and discards memory, maintaining functional correctness by ensuring all participating PEs execute identical load sequences.
- **Cross-PE Reduction** (`direction` attribute in `tl.store`): Extends `tl.store` to send computed results directly to neighboring PEs, enabling collaborative computation patterns where multiple program instances cooperate to produce results. This mechanism supports efficient row-wise and column-wise reductions across PE arrays.
- **Runtime Barriers** (`tl.pe_runtime_barrier()`): Introduces runtime synchronization across all PEs, enabling cross-PE reduction mechanisms and eliminating kernel splitting for explicit synchronization. This primitive maps directly to `libjit_fba_runtime_barrier()`, requiring careful placement to execute exactly once per PE in the physical grid.
- **Explicit Tensor Copies** (`tl.copy()`): Creates deep tensor copies facilitating producer-consumer synchronization between MTIA cores. The compiler automatically detects data race conditions and fails compilation if copy operations prove insufficient for correctness.

Knowledge Injection Mechanism. The MTIA hardware subtree (`hardware/mtia/`) in the persistent knowledge base contains multiple documents spanning architectural overviews, language extensions, optimization

patterns, and complete code examples. When the deep search sub-agent receives queries targeting MTIA hardware, it retrieves relevant documentation—libdevice API references for activation functions, cross-PE communication patterns for multi-PE kernels, custom type definitions for runtime structure manipulation. This retrieved content enters the LLM’s context window during prompt synthesis, effectively teaching the model MTIA-specific programming idioms absent from pretraining data. The context memory sub-agent further refines retrieval based on runtime feedback: compilation errors citing undefined MTIA primitives trigger retrieval of language extension documentation; profiling results indicating suboptimal SFU utilization trigger retrieval of libdevice mapping tables.

This knowledge injection strategy proves critical for MTIA kernel generation quality. Without MTIA-specific documentation in context, LLMs generate standard Triton code targeting GPU semantics, producing compilation failures or functionally incorrect kernels when executed on MTIA hardware. With systematic knowledge base retrieval, KernelEvolve successfully generates production-grade MTIA kernels leveraging hardware-specific features—SFU operations, inter-PE communication, dual-core synchronization—approaching hand-optimized performance while maintaining the productivity benefits of automated generation. This approach generalizes to emerging accelerator architectures: as new hardware platforms enter production, corresponding documentation injected into the knowledge base enables immediate LLM-based kernel generation without model retraining.

3.3 File and Code Search

The deep search and context memory sub-agents operationalize their retrieval strategies through a unified code search interface implemented via Model Context Protocol (MCP) tools [Anthropic 2024]. This interface leverages Meta’s production code search infrastructure—distributed systems (BigGrep, Glean [Marlow and Iborra 2024]) operating over fbsource, Meta’s monolithic repository analogous to Google’s monorepo [Potvin and Levenberg 2016]—capable of querying billions of lines of code providing millisecond-latency retrieval through pre-built indices and optimized search algorithms. The search infrastructure automatically detects and follows code references embedded within knowledge base documentation: when retrieved content contains fbcode file paths or repository links, the tools automatically trigger secondary searches retrieving the referenced implementation code. This automatic dereferencing mechanism bridges curated documentation and production codebases, allowing sub-agents to seamlessly traverse from abstract optimization guidelines to concrete implementation examples without manual intervention.

Search Modalities. The code search interface exposes three search modes via MCP tool invocations: STRMATCH executes exact string matching for locating specific identifiers (function names, API calls, hardware operations); REGEX performs pattern-based queries for matching structural patterns (class definitions, function signatures, optimization templates); FILENAME locates files matching path patterns (hardware-specific modules, configuration files, test implementations). These modes execute against both the knowledge base filesystem and distributed production code indices, providing millisecond-latency retrieval across documentation and implementation corpora.

Result Integration. Search results return as file paths with code snippets including contextual lines (default: 1 leading, 1 trailing). When automatic dereferencing occurs, both the knowledge base documentation and retrieved production code integrate into prompt synthesis, providing the LLM with complementary information: abstract optimization principles from curated documentation alongside concrete implementation patterns from production systems. This enables generation of kernels satisfying theoretical optimization criteria while conforming to organizational coding conventions and platform-specific idioms observed in deployed infrastructure.

3.4 Evaluation and Tooling Framework

3.4.1 Kernel Output Format

KernelEvolve generates kernel implementations conforming to a standardized dual-implementation interface enabling automated correctness validation and performance benchmarking. Each generated artifact comprises three components: a PyTorch baseline, an optimized Triton kernel, and input data generation, structured to support systematic evaluation and downstream compilation integration.

Dual Implementation Interface. Generated kernels provide two `nn.Module` implementations with identical signatures¹.

```
import torch
import triton
import triton.language as tl
# from triton_mtia.python.mtia.eager import mtia_triton_launcher
# mtia_triton_launcher.init() # REQUIRED for MTIA execution

class PytorchModel(nn.Module):
    def forward(self, *args) -> torch.Tensor:
        return pytorch_impl(*args) # Baseline: standard PyTorch ops (torch.matmul, torch.sum)

@triton.jit
def optimized_kernel(...):
    pass # Low-level Triton implementation (tl.load, tl.store, tl.dot)

class TritonModel(nn.Module):
    def forward(self, *args) -> torch.Tensor:
        return kernel_wrapper(*args) # Launch Triton kernel with grid configuration

def get_inputs() -> List[Tuple[torch.Tensor, ...]]:
    # Generate test cases across multiple scales
    return [(torch.randn(N, N, device="cuda"), ...)]
    for N in [512, 1024, 2048, 4096]
```

The PyTorch baseline prioritizes correctness over performance, providing ground truth for numerical validation. The Triton implementation consists of a `@triton.jit` kernel, a wrapper managing grid configuration and kernel launch, and an `nn.Module` encapsulation. Input generation produces test cases spanning diverse sizes, exposing performance characteristics under varying computational and memory pressure.

Design Rationale. The `nn.Module` structure enables integration with PyTorch’s `torch.compile` infrastructure, supporting hybrid optimization strategies combining compiler-driven graph transformations with hand-optimized kernels. Standardized interfaces enable automated evaluation: correctness validation executes both implementations on identical inputs, comparing outputs via `torch.allclose()` with precision-dependent tolerances; performance profiling measures execution time and computes speedup ratios. This structured evaluation framework transforms kernel assessment from manual validation to systematic automation, enabling tree search to evaluate thousands of variants while maintaining correctness guarantees.

3.4.2 AI Hardware Interpreters

Generated Triton kernels require execution on target hardware platforms for correctness validation and performance profiling. To enable automated evaluation across heterogeneous accelerators (NVIDIA GPUs, AMD GPUs, MTIA), KernelEvolve establishes dedicated interpreter environments providing standardized execution contexts with complete software stacks, compilation toolchains, and runtime dependencies for each hardware target.

Bento-Based Interpreters. Meta’s Bento platform—the standard Jupyter notebook environment—bundles external libraries (PyTorch, Triton, CUDA/ROCm) with internal frameworks including hardware-specific software stacks (MTIA runtime, vendor-specific compilers) and build systems. KernelEvolve configures three hardware-specific interpreters: `meta_kernel_gpu_interpreter` (NVIDIA), `meta_kernel_amd_interpreter` (AMD), and `meta_kernel_mtia_interpreter` (MTIA). Each interpreter encapsulates platform-specific compilation toolchains (TritonGPU/AMDGPU/MTIA-MLIR backends), runtime libraries (`mtia_triton_launcher` for MTIA, `cuDNN` for NVIDIA), and profiling tools (NCU, Proton).

¹For training operators requiring gradient computation, both implementations must additionally provide `backward()` methods with matching input/output signatures for gradient propagation.

Node	Last Release ⓘ	Running Release ⓘ																				
			R54	R53	R52	R51	R50 ⚠	R49 ⚠	R48 ⚠	R47 ⚠	R46 ⚠	R45 ⚠	R44 ⚠	R43	R42	R41	R40	R39 ⚠	R38 ⚠	R37 ⚠	R36 ⚠	R35 ⚠
📦 Graft Node	✓ R54 1s		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
📦 Contbuild Tracking Node (bento_kernel_meta_kernel_mtia_interpreter)	✓ R54 0s		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
📦 fbpkg Build Node (bento_kernel_meta_kernel_mtia_interpreter)	✓ R54 14m		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
📦 kernel-startup	✓ R54 9m		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
📦 blocking-notebook-steps	✓ R54 2m		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓										
📦 Tag	✓ R54 1m		✓	✓	✓	✓								✓	✓	✓	✓	✓	✓	✓	✓	✓
📦 nonblocking-notebook-steps	🛡️ ✓ R54 2m		✓	✓	✓	✓																

Figure 8 Continuous deployment pipeline for KernelEvolve interpreter environments via Meta’s Conveyor system. The pipeline monitors dependency updates across Triton compiler backends, hardware runtime libraries, and build systems, automatically triggering daily rebuilds and deployments. Green checkmarks indicate successful releases, red crosses denote failed builds, and yellow warnings flag non-critical issues. The `bento_kernel_meta_kernel_mtia_interpreter` node shows regular deployment cadence with occasional build failures, demonstrating automated recovery and continuous integration across multiple release candidates (R54, R53, etc.).

Automated Deployment. Interpreters integrate with Meta’s Conveyor continuous deployment system [Grubic et al. 2023], which monitors dependency updates and automatically publishes new versions on regular schedules (Figure 8). When underlying components update (Triton compiler, runtime libraries, build systems), Conveyor rebuilds and deploys interpreter packages with current dependencies. This eliminates manual environment maintenance: kernel artifacts submitted to interpreters execute directly against up-to-date software stacks. Interpreter isolation ensures reproducible profiling across tree search iterations.

This infrastructure abstracts hardware-specific complexities from kernel generation, enabling unified evaluation interfaces while maintaining platform-specific optimization capabilities.

3.4.3 Evaluation Code Generation

Kernel candidates generated by tree search conform to the standardized interface specification (`PytorchModel`, `TritonModel`, `get_inputs()`), but require evaluation harness instrumentation for automated profiling. KernelEvolve employs multiple profiling tools targeting different analysis objectives: TritonBench [Meta 2025a] validates correctness through numerical comparison against PyTorch baselines and measures speedup ratios across production input shapes; PyTorch Profiler [Meta 2021] captures system-level execution timelines including kernel launch overhead and host-device synchronization; NCU [Nvidia 2025] provides kernel-level hardware metrics including occupancy, memory throughput, and instruction mix; Proton [Zhou et al. 2025c] delivers intra-kernel instruction-level latency and pipeline behavior; MTIA Insight provides comprehensive MTIA-specific instrumentation: PE utilization, fixed-function accelerator metrics (DPE/SFU/MLU utilization and stall cycles), per-PE CPU runtime, cache analysis (CPU I/D-cache hit rates, branch prediction, LLC behavior), memory bandwidth (LLC/DRAM), and load-store throughput with per-PE read/write counters. KernelEvolve employs a deterministic code generator transforming LLM-generated kernel implementations into platform-specific evaluation scripts invoking these profiling tool APIs (Figure 9).

Multi-Tool Evaluation Harness Generation. The evaluation code generator accepts standardized kernel artifacts as input and produces executable Python scripts for each profiling tool. For TritonBench, the generator creates benchmark harnesses wrapping both `PytorchModel` and `TritonModel` within the `BenchmarkOperator` framework, configuring correctness validation (`baseline=True`) and speedup measurement. For Torch Profiler, generated scripts insert profiler contexts (`torch.profiler.profile()`) around kernel invocations capturing execution traces. For NCU and Proton (via Triton MPP), the generator synthesizes instrumentation invoking respective APIs with hardware-specific configurations. Each generated script imports models from standardized artifacts, instantiates profiler contexts, executes across test cases from `get_inputs()`, and formats results as structured data consumable by the context memory sub-agent.

Interpreter Execution Model. Generated evaluation scripts leverage pre-deployed interpreter environments eliminating compilation overhead. Since hardware interpreters bundle complete toolchains (Triton compilers,

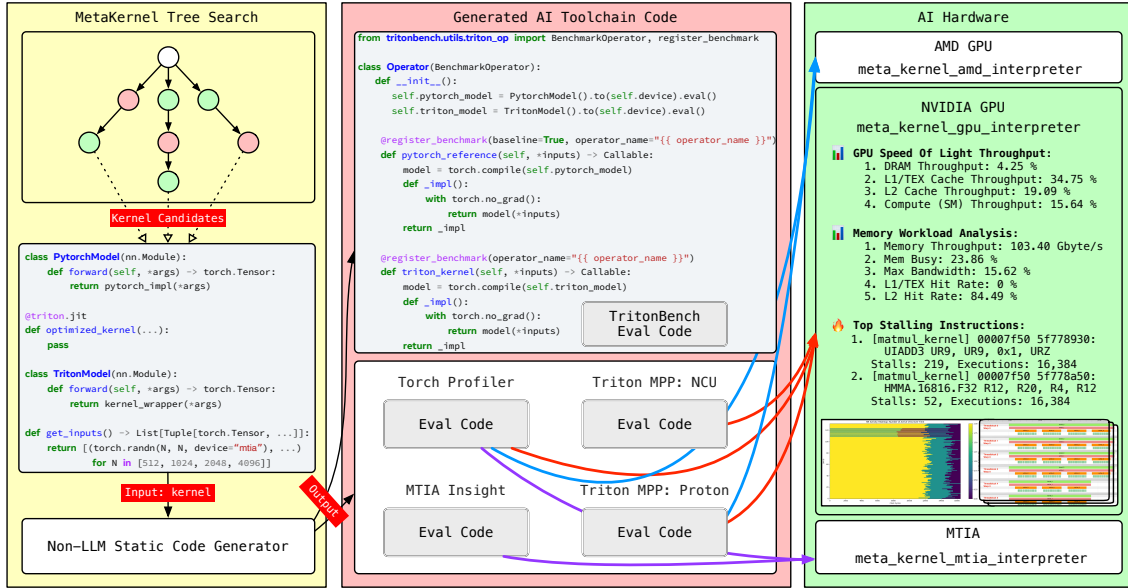


Figure 9 End-to-end evaluation pipeline. Tree search generates kernel candidates with standardized dual implementations (PyTorch baseline, Triton optimized), executed on hardware interpreters (GPU, AMD, MTIA) collecting platform-specific profiling metrics via TritonBench, NCU, MPP, and MTIA Insight. Profiling feedback guides subsequent search iterations.

profiling frameworks, runtime libraries) via Conveyor’s continuous deployment, evaluation code executes immediately without dependency resolution or environment setup. This architectural separation—LLM-generated kernel logic versus deterministically-generated evaluation harness—provides critical benefits: (1) compilation occurs once during interpreter deployment rather than per-kernel evaluation (reducing latency from ≥ 10 minutes to seconds); (2) evaluation code remains consistent across kernel variants, ensuring reproducible profiling; (3) profiling tool APIs update independently through interpreter redeployment without modifying kernel generation prompts.

Figure 9 illustrates this workflow: tree search produces kernel candidates (left panel), the evaluation code generator transforms these into tool-specific harnesses invoking TritonBench, profilers, and hardware-specific instrumentation (center panel), and hardware interpreters execute generated evaluation code collecting platform-specific metrics (right panel) that feed back to guide subsequent tree search iterations.

3.4.4 Unified Profiling: Triton MPP

KernelEvolve integrates Triton MPP (Multi-Pass Profiler) from Meta addressing fundamental GPU profiling challenges. Modern profiling workflows exhibit fragmentation: practitioners orchestrate IR-level tracers, assembly profilers (NCU), and binary instrumentation (NVBit), each with incompatible interfaces and vendor-specific assumptions. Profilers target human interpretation through textual reports and dashboards rather than structured data, forcing brittle text parsing unsuitable for automation. MPP provides a compiler-centric abstraction unifying heterogeneous profiling tools. The framework composes analysis through job graphs: compiler transforms insert MLIR-level instrumentation, profiling passes collect metrics, trace synthesis produces structured output. This proves critical for modern Triton kernels employing TMA operations, warp-specialized pipelines, and overlapped data movement.

Traditional GPU profilers expose asynchronous behavior coarsely, failing to reveal instruction-level overlap between memory and computation. Direct wait insertion perturbs execution: initial synchronization disrupts overlap, cascading timing changes through subsequent operations. MPP addresses this through minimally-invasive profiling: capturing unmodified base traces, applying targeted probe passes isolating single instructions in specific iterations, guarding instrumentation to prevent interference, and fusing results for attribution. This profiles warp-group operations, async copies, and TMA transfers at TTGIR level with negligible perturbation.

MPP integration provides KernelEvolve with structured, instruction-level performance data without vendor-specific parser implementations, transforming profiling from manual orchestration to programmatic composition.

3.4.5 Agentic Debugging in JIT Flow

Beyond profiling metrics, MTIA-Triton provides compiler introspection capabilities critical for kernel debugging and optimization. MTIA-Triton supports optional C++ code emission exposing the compiler’s intermediate representation before final RISC-V binary generation:

```
compiled_kernel = kernel[grid](x, *x.shape, BLOCK_SIZE=1024, cb_multiplier=8, emit_cxx=True)
cpp_source = compiled_kernel.asm["cpp"]
"""
/// This is a generated Triton C++ kernel.
extern "C" bool __mtia_is_core_b();
extern "C" uint64_t __mtia_scoped_block_print_info(uint64_t, uint64_t);
extern "C" v256_float16 __mtia_rvv_init256_fp16(float16_t);
extern "C" void __mtia_adjust_cb_read_pointer(uint32_t, uint32_t);
extern "C" void __mtia_adjust_cb_write_pointer(uint32_t, uint32_t);
...
"""
```

The emitted C++ code reveals low-level MTIA operations including RISC-V vector intrinsics, circular buffer pointer management, SFU initialization, and core affinity queries.

Interactive Debugging Workflow. When generated kernels crash, fail correctness validation, or exhibit unexpected performance, the context memory sub-agent retrieves emitted C++ code alongside error diagnostics. The universal operator analyzes this representation—identifying incorrect buffer management, missing synchronization, or suboptimal SFU usage—and generates corrected C++ implementations. Modified C++ kernels can be tested immediately without full recompilation through MTIA’s replay mechanism:

```
from triton_mtia.python.mtia.eager.debug.replay_cpp import replay_cpp
# Rebuild modified C++ and launch with runtime arguments only
# (constexprs and compiler options already baked in)
replay_cpp(modified_cpp_source, compiled_kernel,
           args=(1, 1, 1, x, *x.shape)) # PID grid + runtime args
```

This rapid iteration cycle—emit C++, modify, replay—eliminates full Triton recompilation overhead (constexpr resolution, MLIR lowering, backend code generation), enabling agents to validate hypothesized fixes within seconds rather than minutes. Compiler introspection transforms opaque execution failures into actionable optimization opportunities grounded in hardware-specific implementation details.

3.4.6 Kernel Evaluation on FaaS

Tree search execution decomposes each node expansion into two phases: kernel generation and kernel evaluation. Generation comprises prompt synthesis, knowledge base retrieval, and LLM invocation—CPU-bound operations requiring no accelerator access. Evaluation executes generated kernels on target hardware through TritonBench correctness validation, Torch Profiler timeline capture, and Triton MPP instruction-level analysis. This workload asymmetry—generation on CPU hosts versus evaluation on AI accelerators—motivates architectural disaggregation enabling independent, non-blocking evaluation execution on remote hardware.

FaaS Integration for Remote Evaluation. Kernel evaluation is an ideal FaaS workload: individual evaluation functions execute independently without inter-function dependencies or communication, unlike serverless databases requiring aggregation coordination [Liao et al. 2023]. KernelEvolve migrates kernel evaluation to Meta’s FaaS (Function-as-a-Service) platform [Sahraei et al. 2023], which abstracts infrastructure complexity

including service routing [Saokar et al. 2023], Twine autoscaling [Tang et al. 2020], and lifecycle management. FaaS runtime auto-generates Thrift server interfaces for evaluation handlers, packaging them as fbpkg distributions with hardware interpreter dependencies. We extended FaaS’s Tasklet resource model from CPU/RAM to include GPU resources, enabling hardware-specific evaluation functions targeting NVIDIA, AMD, and MTIA platforms. When tree search generates a kernel candidate, it asynchronously dispatches evaluation requests to FaaS endpoints corresponding to target hardware. Remote workers load pre-deployed interpreter environments, execute evaluation harnesses, and return structured results (correctness status, speedup ratios, profiling metrics) consumed by the context memory sub-agent.

Benefits and Scalability. Disaggregation addresses resource contention: a single host may run hundreds of generation agents but possess limited accelerators (8 GPUs or 24 MTIA devices per host). Without separation, agents serialize through available hardware—each occupying a device for 8-12 minutes (mostly idle during generation) while other agents queue. FaaS-based evaluation provides: (1) *resource decoupling*—generation agents execute CPU-bound work locally while dispatching evaluation to remote accelerator pools, preventing device occupation during idle generation phases; (2) *elastic capacity*—evaluation distributes across FaaS worker pools with hundreds of GPUs/MTIA devices rather than serializing through local hardware. This architecture maximizes both CPU (generation) and accelerator (evaluation) utilization, eliminating the mismatch between abundant generation parallelism and scarce local hardware resources.

4 OSS Operator Evaluation

Kernel coverage—the availability of optimized implementations for standard operators—is a fundamental prerequisite for deploying models on emerging AI accelerators. Before optimizing for performance, the system must first demonstrate the ability to generate correct kernels across the operator set. This section evaluates KernelEvolve’s end-to-end capability to generate, validate, and benchmark kernels across heterogeneous hardware.

We curate a test suite of 160 ATen operators covering basic computational patterns: element-wise arithmetic (`torch.add`, `torch.div`), transcendental functions (`torch.cos`, `torch.exp`), reductions (`torch.amax`, `torch.allclose`), and activation primitives (`torch.ops.aten.elu`). While these operators are relatively simple, they represent the foundational building blocks required for PyTorch model execution and serve as an end-to-end validation of KernelEvolve’s kernel generation correctness. For each operator, KernelEvolve generates Triton kernel implementations targeting three platforms: NVIDIA H100, AMD MI350, and MTIA v3. Generated kernels are validated against PyTorch reference implementations compiled with `torch.compile`. Numerical equivalence is verified using `torch.allclose` in TritonBench with precision-appropriate tolerances.

KernelEvolve achieves 100% correctness across all 480 operator-platform configurations (160 operators $\times$ 3 platforms). We further validate on KernelBench [Ouyang et al. 2025], achieving 100% pass rate across all three levels: Level 1 (single operators), Level 2 (fused operator patterns), and Level 3 (full model blocks). While KernelBench originally targets CUDA kernel generation, these results demonstrate that KernelEvolve reliably produces numerically correct Triton kernels across diverse architectures and operator complexities—from individual primitives to end-to-end model components—establishing the foundation for addressing the kernel coverage challenge on emerging hardware.

Figure 10 shows optimization trajectories for six operators. The fitness score is defined as the speedup of the generated Triton kernel over the PyTorch reference. The search operates in two phases. In the draft phase (steps 0–10), KernelEvolve generates candidate kernels through independent sampling without feedback. In the tree expansion phase (steps 10–50), each node incorporates execution feedback—profiling data, compilation status, and correctness results—from its ancestors, enabling iterative refinement. The trajectories exhibit operator-dependent behavior. `torch.cos` improves from $2.8\times$ to $3.05\times$ during tree expansion, indicating that feedback-guided search discovers superior implementations. `torch.ops.aten.add.Tensor` shows early-stage improvement ($0.64\times$ to $0.70\times$), demonstrating that iterative refinement benefits even initially suboptimal kernels. `torch.amax` and `torch.div` remain near $1.0\times$ throughout, suggesting limited optimization headroom for these operators. Four of six operators achieve fitness scores exceeding $1.0\times$, confirming that KernelEvolve-generated kernels can outperform compiler-generated baselines.

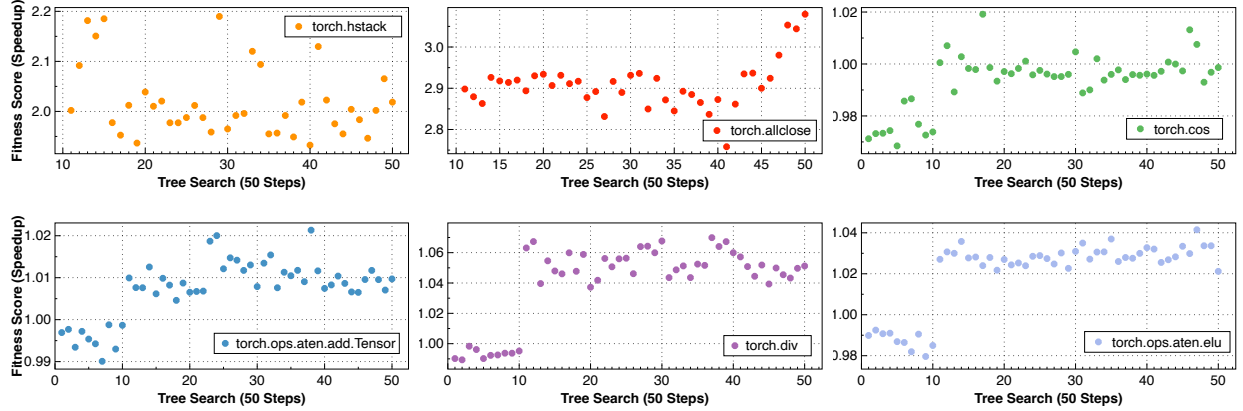


Figure 10 Fitness score trajectories during KernelEvolve’s tree search optimization for 6 representative ATen operators. The x-axis denotes search steps (50 total), and the y-axis shows the fitness score defined as the speedup ratio of the generated Triton kernel over the PyTorch baseline. The first 10 steps correspond to the draft phase (repeated sampling without memory context), while subsequent steps represent tree expansion with execution feedback.

These basic ATen operators serve primarily to validate KernelEvolve’s end-to-end correctness rather than to demonstrate optimization potential. As fundamental primitives, they offer limited headroom for improvement. In the following section, we evaluate KernelEvolve on high-level ops operators, which compose multiple ATen primitives with ops-specific logic, exhibiting unique fusion opportunities and memory access patterns that yield substantially larger optimization potential and direct business impact.

5 Monetization Case Study

Figure 4 presents KernelEvolve’s performance across production workloads, achieving 1.2-17 $\times$ speedups over PyTorch baselines. We present detailed analysis of representative kernels: 1D convolution in convolutional transformers, operator fusion in WuKong’s Optimized FM and InterFormer’s PFFN, and data preprocessing operators (MapId, MBDT, Batch Event Truncate) across heterogeneous accelerators.

Beyond these detailed case studies, several kernels achieve substantial speedups through more straightforward optimizations: expanded autotuning search spaces exploring block sizes and pipeline configurations, platform-specific compilation flags (MTIA’s `cb_multiplier`), and memory access improvements (cache modifiers, alignment hints). While individually less sophisticated, these optimizations demonstrate KernelEvolve’s ability to systematically explore configuration spaces that manual development often overlooks.

5.1 Convolutional Transformer

Inspired by CNN Inceptions [Szegedy et al. 2015] and convolution-augmented transformers [Gulati et al. 2020], the Convolutional Transformer architecture combines convolutional and transformer components to capture both local and global patterns in user sequential events for large-scale recommendation systems.

The core of this architecture is a stack of 1D convolutional layers, which serve as the receptive field. Through multi-scale sliding windows with varying kernel sizes and strides, these conv1d layers compress long event sequences into shorter segment representations, extracting hierarchical patterns at different granularities. Given that conv1d operations dominate the computational workload, kernel-level optimization is critical for deployment at scale. To address this, we employ KernelEvolve to automatically generate and tune high-performance kernels on H100 GPUs through iterative refinement.

Baselines and Evaluation Setup. We compare KernelEvolve-generated kernels against two PyTorch baselines on production shapes. The first baseline uses `torch.nn.functional.conv1d` directly. The second—a common optimization technique—reshapes input to 2D with `channels_last` memory format and invokes `torch.nn.functional.conv2d`, mapping to cuDNN’s heavily optimized Tensor Core path for NHWC convolu-

Precision	Tensor Shape ($B \times C_{in} \times C_{out} \times L$)	torch.conv1d (ms)	torch.conv2d (ms)	Triton (ms)	Speedup vs. conv1d	Speedup vs. conv2d
FP16	$64 \times 96 \times 96 \times 200$	0.03050	0.02019	0.01597	1.91×	1.26×
	$128 \times 96 \times 96 \times 200$	0.03840	0.02490	0.01830	2.10×	1.36×
	$256 \times 96 \times 96 \times 200$	0.05318	0.03347	0.02842	1.87×	1.18×
	$512 \times 96 \times 96 \times 200$	0.08646	0.06006	0.04982	1.74×	1.21×
	$1024 \times 96 \times 96 \times 200$	0.17226	0.11299	0.08406	2.05×	1.34×
	$2048 \times 96 \times 96 \times 200$	0.34243	0.24106	0.14864	2.30×	1.62×
	$32 \times 64 \times 64 \times 512^\dagger$	0.02768	0.01779	0.01264	2.19×	1.41×
	$32 \times 256 \times 256 \times 1024^\dagger$	0.07933	0.05485	0.06029	1.32×	0.91×
FP32	$64 \times 768 \times 768 \times 1024^\dagger$	0.71549	0.55354	1.12784	0.63×	0.49×
	$64 \times 96 \times 96 \times 200$	0.03501	0.02531	0.02186	1.60×	1.16×
	$128 \times 96 \times 96 \times 200$	0.04630	0.03248	0.03510	1.32×	0.93×
	$256 \times 96 \times 96 \times 200$	0.07168	0.05712	0.05789	1.24×	0.99×
	$512 \times 96 \times 96 \times 200$	0.15030	0.11517	0.11219	1.34×	1.03×
	$1024 \times 96 \times 96 \times 200$	0.32077	0.24269	0.19725	1.63×	1.23×
	$2048 \times 96 \times 96 \times 200$	0.61411	0.46384	0.35594	1.73×	1.30×
	$32 \times 64 \times 64 \times 512^\dagger$	0.02730	0.01978	0.01571	1.74×	1.26×
	$32 \times 256 \times 256 \times 1024^\dagger$	0.13469	0.10326	0.13501	1.00×	0.77×
	$64 \times 768 \times 768 \times 1024^\dagger$	1.26237	1.04234	2.64502	0.48×	0.39×

Yellow : production configuration. Purple † : randomly selected shapes (not optimization target).

Table 3 Conv1d kernel performance: KernelEvolve-generated Triton kernel vs. PyTorch conv1d and conv2d baselines. The kernel is optimized for production ads ranking shapes (highlighted in yellow), achieving strong speedups. Performance on other shapes (highlighted in purple) varies: similar shapes benefit from the optimization, while out-of-distribution shapes show degraded performance.

tions. Table 3 evaluates performance across batch sizes with FP16 (serving) and FP32 (training) precision, verified with $\text{atol}=10^{-4}$, $\text{rtol}=5 \times 10^{-4}$.

Performance Results. On production shape ($B \times C_{in} \times C_{out} \times L$) = (2048, 96, 96, 200), KernelEvolve achieves 2.30× speedup over conv1d and 1.62× over the optimized conv2d baseline in FP16, with consistent gains across batch sizes (1.74-2.30× vs. conv1d). For FP32 training workloads, speedups reach 1.73× over conv1d and 1.30× over conv2d. The generated kernel is deliberately specialized: on out-of-distribution shapes (e.g., $64 \times 768 \times 768 \times 1024$), it underperforms baselines (0.49-0.63×), confirming that optimization targets production distributions rather than arbitrary inputs.

Kernel Fusion as Primary Optimization. Figure 11 and Table 4 reveal the source of these improvements through execution trace analysis. PyTorch conv1d launches five separate kernels: multiple layout transformations (nchwToNhwKernel, nhwcToNchwKernel), Tensor Core GEMM (sm90_xmma_fprop), and a Triton-generated fusion kernel. Each launch incurs synchronization overhead and intermediate memory traffic. The conv2d baseline reduces this to four kernels via optimized NHWC paths but still requires separate operations for layout manipulation and computation.

KernelEvolve fuses the entire operation into two kernels: weight preparation and the main convolution kernel. The Triton Conv1d kernel outperforms the PyTorch Conv2d workaround by eliminating memory layout conversions. The Conv2d approach launches four auxiliary kernels for unsqueeze, channels-last conversion, convolution, and squeeze operations—each requiring a full tensor pass and incurring significant global memory traffic. In contrast, Triton launches only two kernels: a lightweight weight-packing step and the fused convolution, operating directly on the native 1D layout. While cuDNN’s implicit GEMM achieves high compute efficiency for the convolution itself, Triton delivers better end-to-end performance by avoiding redundant layout transformations and their associated memory overhead.

Complementary Optimizations. Beyond fusion, the generated kernel employs architectural optimizations discovered through search. Expanded autotuning explores over 20 configurations across block sizes, warp counts, and pipeline stages, tailored to input dimensions and convolution parameters. A 3D grid launch

ID	Estimated Speedup	Function Name	Torch.Conv1d	Demangled Name	Duration [ms]	
0	17.81	nchwToNhwKernel		..nchw2nhw...	92.64	
1	526.36	nchwToNhwKernel		..nchw2nhw...	4.70	
2	45.13	sm90_xmma_fprop_implicit_gemm_bf16bf16_bf16f32_f32_nhwckrsc...		sm90_xmm...	93.57	
3	17.91	nchwToNchwKernel		..nhwc2nch...	102.98	
4	13.82	triton_poi_fused_convolution_0		triton_poi_fu...	63.52	

ID	Estimated Speedup	Function Name	Torch.Conv2d	Demangled Name	Duration [ms]	Runtime Improv
0	31.69	triton_poi_fused__to_copy_unsqueeze_0		triton_poi_fu...	64.16	2
1	337.26	triton_poi_fused__to_copy_convolution_unsqueeze...		triton_poi_fu...	3.87	1
2	45.17	sm90_xmma_fprop_implicit_gemm_bf16bf16_b...		sm90_xmm...	93.73	4
3	13.28	triton_poi_fused__to_copy_convolution_unsqueeze...		triton_poi_fu...	63.20	

ID	Estimated Speedup	Function Name	Triton Kernel	Demangled Name	Duration [ms]	Runtime Improv
0	675.45	pack_conv1d_weight_kernel		pack_conv1...	6.08	4
1	37.99	conv1d_gemm_kernel		conv1d_ge...	197.50	5

Figure 11 Profiling traces comparing conv1d implementations on production shape. PyTorch conv1d (top) launches five separate kernels including layout transformations and GEMM. PyTorch conv2d (middle) reduces to four kernels via optimized NHWC paths. KernelEvolve (bottom) fuses operations into two kernels with cross-operation fusion. Note that durations shown in the profiling trace include profiling overhead and do not represent actual kernel latency.

Implementation	Kernel	Operation
torch.nn.Conv1d	nchwToNhwKernel nchwToNhwKernel sm90_xmma_fprop_implicit_gemm nchwToNchwKernel triton_poi_fused_convolution_0	Convert input NCHW → NHWC Convert weights NCHW → NHWC Convolution (cuDNN implicit GEMM) Convert output NHWC → NCHW Bias addition / post-processing
torch.nn.Conv2d	triton_poi_fused_to_copy_unsqueeze_0 triton_poi_fused_to_copy_convolution_unsqueeze_1 sm90_xmma_fprop_implicit_gemm triton_poi_fused_to_copy_convolution_unsqueeze_2	Layout conversion (unsqueeze + channels-last) Weight preparation (unsqueeze) Convolution (cuDNN implicit GEMM) Post-processing (squeeze 4D → 3D)
KernelEvolve	pack_conv1d_weight_kernel	Weight packing for GEMM-style access
Triton Conv1d	conv1d_gemm_kernel	Fused GEMM-style convolution

Table 4 Kernel breakdown comparison for conv1d implementations. PyTorch conv1d incurs significant layout conversion overhead. PyTorch conv2d reduces conversions through optimized NHWC paths. Triton conv1d eliminates redundant transformations through kernel fusion.

parallelizes grouped convolution channels, eliminating inter-group dependencies. Double-buffered execution prefetches the next data blocks while computing current blocks, overlapping memory access with Tensor Core operations. Differentiated cache modifiers optimize memory hierarchy usage (.ca for streaming activations, .cg for reused weights).

Search-Based Discovery. Figure 12 visualizes the optimization trajectory over 300 search steps, where the fitness score equals 1/latency (higher is better). Green nodes indicate successful generations; red nodes indicate compilation or correctness failures. Initial draft phases achieve fitness scores around 2000. As search progresses with accumulated execution feedback, scores improve systematically—reaching 4000, then 5000, and ultimately converging to 6889. This trajectory demonstrates that graph-based search with performance-guided selection discovers increasingly efficient implementations through inference-time scaling, automatically identifying the fusion strategies and tiling configurations that manual development would require weeks to explore.

Appendix A provides source code for the PyTorch conv1d and conv2d baselines, the KernelEvolve-generated Triton kernel, and TritonBench scripts for accuracy and speedup evaluation.

5.2 Convolution on Heterogeneous Hardware

The diversity of hardware vendors and generations in large-scale model inference environments poses significant challenges for both enablement and optimization. KernelEvolve’s graph-based search and retrieval-

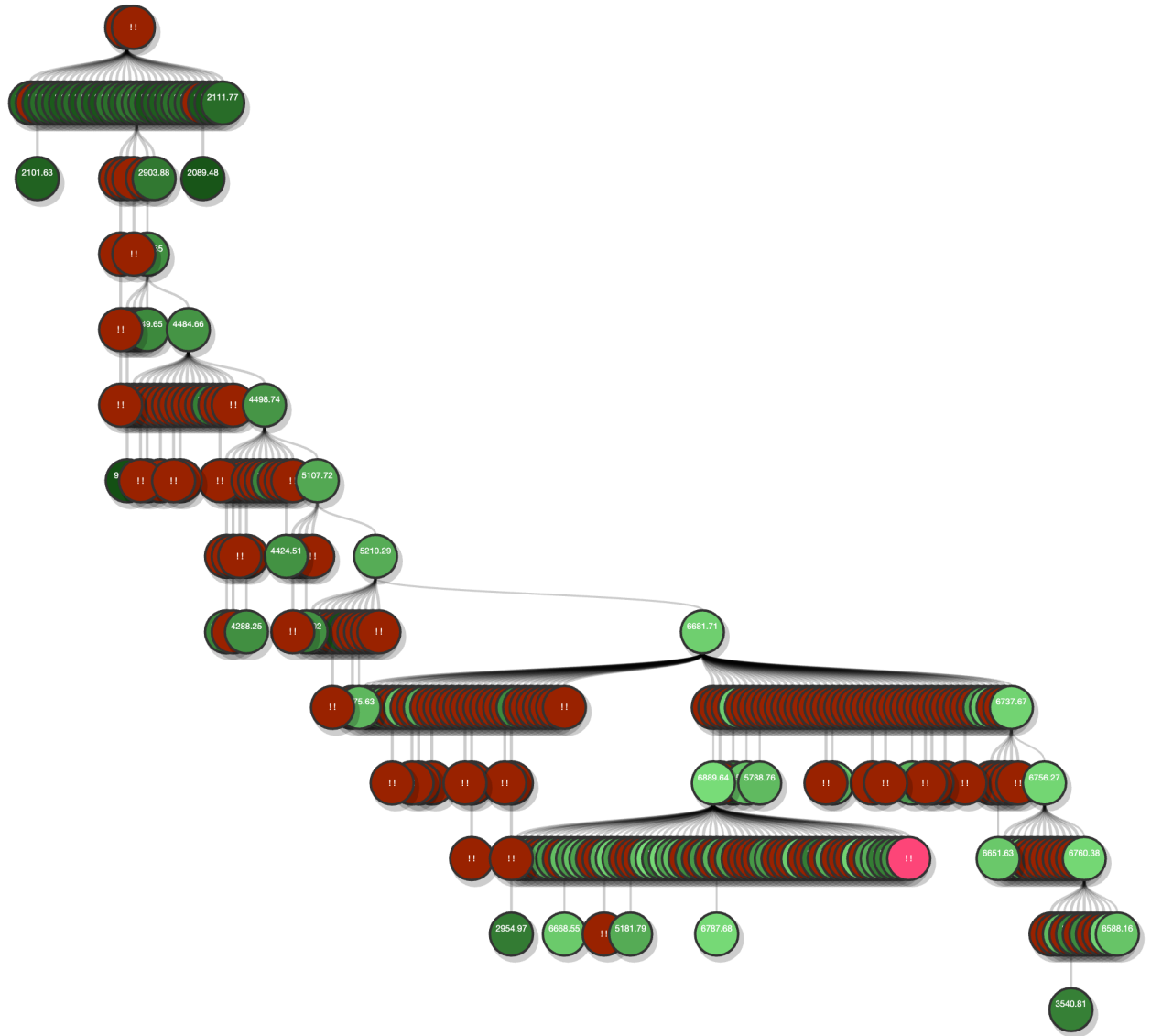


Figure 12 Search tree visualization for conv1d kernel generation over 300 steps. Green: successful generation; Red: compilation/correctness failures.

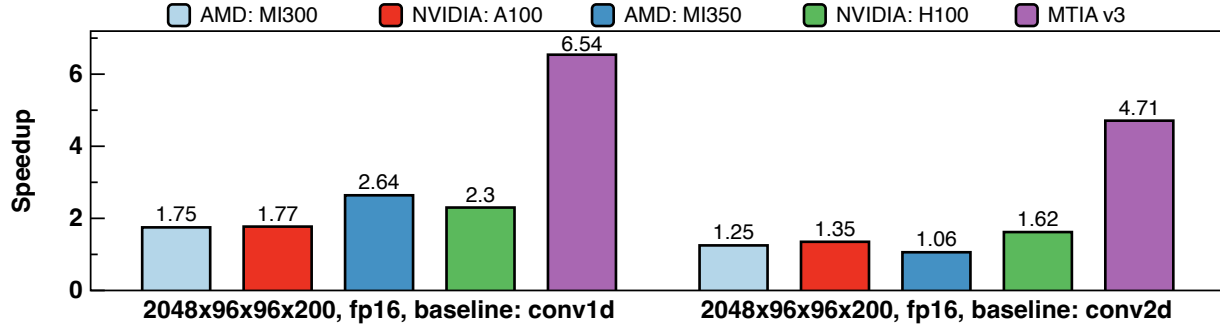


Figure 13 KernelEvolve-generated kernels compared against PyTorch conv1d and optimized conv2d baselines, demonstrating up to $6.22\times$ speedup across NVIDIA, AMD, and MTIA architectures.

augmented prompting are specifically designed to accommodate this heterogeneity, generating optimized kernels for AMD, NVIDIA, and MTIA accelerators from unified operator specifications. To validate cross-platform effectiveness, we evaluate the conv1d kernel from convolutional transformers (Section 5.1) across five hardware platforms spanning three vendors and multiple generations.

Cross-Platform Baseline Comparison. Figure 13 compares KernelEvolve-generated kernels against two PyTorch baselines on production shape $(B \times C_{in} \times C_{out} \times L) = (2048, 96, 96, 200)$ with FP16 precision. The first baseline uses `torch.nn.functional.conv1d` directly. The second baseline employs a common optimization technique: reshape the input to 2D with `channels_last` memory format and invokes `torch.nn.functional.conv2d`, which maps to cuDNN’s heavily optimized Tensor Core path for NHWC convolutions. While mathematically equivalent to conv1d, this conv2d approach often provides superior performance on modern GPUs by exploiting vendor-optimized libraries.

KernelEvolve-generated kernels achieve consistent speedups over the conv1d baseline across all platforms: $1.75\times$ on AMD MI300, $2.30\times$ on NVIDIA H100, $2.54\times$ on AMD MI350, $1.77\times$ on NVIDIA A100, and $6.54\times$ on MTIA v3. Performance relative to the optimized conv2d baseline varies by platform: NVIDIA GPUs show modest improvements ($1.62\times$ on H100, $1.35\times$ on A100), reflecting cuDNN’s maturity on these architectures. AMD platforms demonstrate smaller gains ($1.25\times$ on MI300, $1.06\times$ on MI350). MTIA v3 achieves the largest speedup at $4.71\times$ over conv2d, demonstrating that KernelEvolve’s automated synthesis can effectively target custom accelerator architectures where vendor library coverage is less mature.

Hardware-Specific Optimization Strategies. The performance variations across platforms reflect fundamental architectural differences that KernelEvolve’s knowledge base encodes. On NVIDIA GPUs, generated kernels exploit Tensor Core operations through careful tile sizing and memory layout transformations. AMD platforms benefit from Infinity Cache-aware tiling that maximizes on-chip data reuse. MTIA kernels leverage specialized function units and inter-PE communication primitives absent from GPU programming models (Section 3.2.3). Critically, these hardware-specific optimizations emerge through systematic search guided by platform-specific documentation retrieved from the knowledge base—rather than manual per-platform tuning.

Portability vs. Performance Trade-offs. The results illustrate a fundamental challenge in heterogeneous deployment: optimizations targeting one platform may not transfer to others. KernelEvolve addresses this challenge through shape-aware dispatch: for each platform, the system generates and validates platform-specific kernels during an offline optimization phase, then deploys the highest-performing variant while maintaining fallback paths to vendor libraries (conv1d/conv2d) when generated kernels underperform. This architecture ensures that automated synthesis delivers performance improvements where possible without risking regressions, enabling safe production deployment across diverse accelerator fleets.

5.3 WuKong and InterFormer: Kernel Fusion

5.3.1 Optimized FM in WuKong

Optimized FM is a core computational primitive in Meta’s Wukong recommendation model [Zhang et al. 2024] (Section 3.6). In factorization machine-based architectures, computing the pairwise dot product $XX^\top$ has $O(N^2D)$ complexity, prohibitive for real-world datasets with thousands of features. Wukong exploits the low-rank property of $XX^\top$ by introducing a learnable projection matrix $Y \in \mathbb{R}^{N \times K}$ where $K \ll N$, reducing output dimensionality from $N \times N$ to $N \times K$. Leveraging associativity, the computation reorders to:

$$\text{out} = X \cdot (X^\top Y) \quad (1)$$

```
def pytorch_ref_impl(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    """
    Reference PyTorch native implementation of Optimized FM kernel.
    """
    xty = torch.bmm(x.permute(0, 2, 1), y) # (B, D, N) @ (B, N, K) = (B, D, K)
    return torch.bmm(x, xty)               # (B, N, D) @ (B, D, K) = (B, N, K)
```

where $X \in \mathbb{R}^{B \times N \times D}$ and $Y \in \mathbb{R}^{B \times N \times K}$. Computing $X^\top Y$ first reduces complexity from $O(N^2D)$ to $O(NKD)$. This two-stage batched matrix multiplication presents a fusion opportunity: the intermediate result $X^\top Y \in \mathbb{R}^{B \times D \times K}$ can remain in registers or shared memory, eliminating global memory round-trips.

Production Configuration. We evaluate on production shapes extracted from a deployed Wukong variant: $(B, N, D, K) \in \{(1024, 24, 224, 2198), (1024, 40, 224, 448), (1024, 48, 224, 448)\}$. The PyTorch baseline with `torch.compile` generates two separate `extern_kernels.bmm` calls, each independently loading inputs from HBM, performing computation, and writing results back—missing the fusion opportunity between operations.

Kernel Optimization Strategy. KernelEvolve generates a fused Triton kernel exploiting two key optimizations:

(1) *Operator Fusion Eliminating Intermediate Materialization.* The PyTorch baseline executes two independent matrix multiplications with an intermediate HBM round-trip. KernelEvolve fuses both operations into a single kernel: inputs are loaded once, the intermediate result $X^\top Y$ remains in SRAM throughout computation, and only the final output writes to HBM. This reduces memory traffic by approximately $2\times$ —eliminating one full read-write cycle of the intermediate tensor.

(2) *Shape-Specific Tiling for SRAM Residency.* Rather than using PyTorch’s standardized autotuning templates, KernelEvolve generates custom tiling configurations tailored to production shapes. The kernel decomposes inputs into tiles sized to fit the complete computation chain (both multiplications) within SRAM capacity. For production configurations where $B = 1024$, $D = 224$ remain fixed, tile dimensions are optimized to maximize SRAM utilization while ensuring the fused operation executes entirely on-chip.

Performance Analysis. Figure 14 analyzes speedup across production shape variations. The left panel shows speedup as a function of batch size B for three representative configurations. The kernel achieves $3.6\text{--}3.9\times$ speedup for small N (24 features), maintaining stable performance as batch size scales from 128 to 2048. For medium N (40, 48 features), speedup ranges from $2.1\text{--}3.0\times$ at small batch sizes, gradually decreasing to $2.2\text{--}2.3\times$ as B increases—reflecting the trade-off between fusion benefits and tiling overhead.

The right panel examines speedup as a function of output dimension K with fixed $B = 1024$, $D = 224$. Small N values (24-32 features) maintain $3.0\text{--}3.5\times$ speedup across the entire K range (256-2304), demonstrating robust performance. Medium N values (40-64 features) achieve $2.0\text{--}2.5\times$ speedup, while larger N (96-256 features) show diminishing returns, approaching $1\times$ as N increases. This degradation occurs because larger feature counts require more tiles to fit in SRAM—as the number of tiles grows, the overhead of tile management and accumulation eventually surpasses the benefits of on-chip computation, making direct HBM execution competitive.

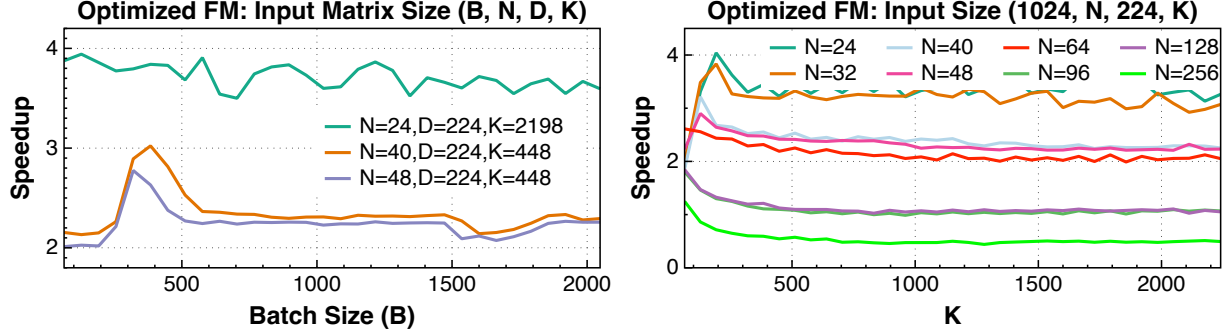


Figure 14 Optimized FM speedup on production shapes. Left: Performance across batch sizes for three representative (N, D, K) configurations extracted from deployed Wukong models. Right: Speedup variation with output dimension K at fixed batch size $B = 1024$, showing strong performance for small-to-medium feature counts ($N \leq 64$) and degradation for larger N where tiling overhead dominates.

Deployment Strategy. The generated kernel demonstrates consistent $2\text{-}4\times$ speedups on production shapes where $N \leq 64$, covering the majority of deployed Wukong variants. For configurations with larger feature counts where tiling overhead dominates, the system falls back to PyTorch’s unfused baseline. This shape-specific dispatch ensures performance gains on target workloads without risking regressions on out-of-distribution inputs. The results validate that KernelEvolve’s search-based optimization can discover fusion strategies and tiling configurations tailored to production distributions, achieving competitive performance with expert manual implementations while reducing development time from weeks to hours.

5.3.2 PFFN in InterFormer

Personalized FeedForward Network (PFFN) is a key component of the InterFormer architecture in Meta’s ads ranking system [Zeng et al. 2025]. In recommendation models, user behavior sequences are inherently noisy—users browse items randomly, making pure sequential modeling ineffective. InterFormer addresses this by enabling bidirectional information flow between non-sequential features (e.g., user demographics) and sequential features (e.g., browsing history).

Module Structure. The PFFN module comprises five operations executed sequentially: (1) a feed-forward neural network (batched matrix multiplication with bias), (2) GELU activation, (3) root-mean-square normalization (RMSNorm), (4) another feed-forward layer, and (5) final RMSNorm. This operator chain processes tensors $X \in \mathbb{R}^{B \times N \times D}$ with weight matrices $W_1 \in \mathbb{R}^{B \times D \times K}$ and $W_2 \in \mathbb{R}^{B \times K \times D}$, where B denotes batch size, N sequence length, D input dimension, and K hidden dimension.

Production Configuration. We evaluate on shapes extracted from deployed InterFormer models: $(B, N, D, K) \in \{(1024, 200, 256, 160), (1024, 200, 192, 96), (1024, 400, 256, 160), (1024, 150, 96, 192)\}$. Production deployments exhibit consistent patterns with $B = 1024$, $N \in [150, 400]$, $D \in [96, 256]$, and $K \in [96, 256]$. The PyTorch baseline with `torch.compile` generates two separate kernels: (1) `extern_kernels.bmm` for matrix multiplication (single pass: load inputs, compute, write output), and (2) a two-pass fused Triton kernel `triton_per_fused_rms_norm_add_gelu` where the first pass loads data to perform bias addition and accumulate RMSNorm statistics, and the second pass reloads data to apply normalization. This results in three total memory round-trips: one for BMM and two for the fused operations. While PyTorch exploits fusion opportunities among element-wise operations, the multi-pass execution and kernel separation incur redundant memory traffic.

Kernel Optimization Strategy. KernelEvolve generates two kernel variants targeting different operator chains: (1) fusing feed-forward network with RMSNorm, and (2) fusing feed-forward network, GELU, and RMSNorm. We select the highest-performing variant for production deployment, evaluating on FP16 precision matching production serving requirements. The generated kernel achieves performance improvements through two key optimizations:

(1) *Shape-Specific Tiling for Target Distributions.* KernelEvolve’s search process incorporates production input

shape ranges during kernel generation. The generated kernel employs customized tiling configurations that maximize SRAM utilization for target dimensions—in contrast to PyTorch’s templated BMM kernel using generic tiling heuristics. For production shapes where $D \in [96, 256]$ and $K \in [96, 256]$, the specialized tiling ensures tiles remain SRAM-resident throughout computation, avoiding HBM fallback that occurs with one-size-fits-all tile sizes.

(2) *Cross-Operation Tile Reuse*. KernelEvolve generates a unified single-pass kernel that loads tiles once, performs the complete operator chain (matrix multiplication, bias addition, GELU, RMSNorm) while data resides in SRAM, and writes final results to HBM—requiring only one load and one write per tile. The PyTorch baseline executes PFFN through two separate kernels with three total passes: (1) the first kernel (`extern_kernels.bmm`) loads inputs, performs matrix multiplication, and writes intermediate results; (2-3) the second kernel (`triton_per_fused_rms_norm_add_gelu`) executes in two passes—the first pass reloads data to perform bias addition and accumulate RMSNorm statistics, the second pass reloads data again to apply normalization.

Performance Analysis. Figure 15 analyzes speedup across production shape variations. The left panel shows performance as a function of batch size B for five production configurations extracted from deployed InterFormer models. Peak speedups of 2.0-2.6 $\times$ occur at small batch sizes ($B \leq 256$), where the fused kernel’s reduced memory traffic dominates performance. As batch size increases beyond 512, speedup stabilizes at 1.2-1.4 $\times$ across all configurations. This convergence reflects a fundamental trade-off: larger batches amortize kernel launch overhead for both optimized and baseline implementations, reducing the relative advantage of fusion as compute-to-memory ratio increases. The configuration ($N = 200, D = 256, K = 160$)—representative of high-dimensional production embeddings—maintains consistent 1.2 $\times$ speedup at large batch sizes, validating robust performance on primary deployment targets.

The right panel examines speedup as a function of input dimension D with fixed $B = 1024, K = 256$ across varying sequence lengths $N \in [150, 400]$. Performance exhibits non-monotonic behavior: speedup peaks at 1.6-1.9 $\times$ for small D ($D \leq 100$), drops to a local minimum of 1.1-1.2 $\times$ around $D = 200$, then recovers to 1.2-1.4 $\times$ for larger dimensions ($D > 200$). This pattern arises from the interplay between tile size and SRAM capacity. At small D , tiles fit comfortably in SRAM enabling effective fusion; at intermediate $D \approx 200$, tile dimensions approach SRAM limits causing partial spilling that negates fusion benefits; at large $D > 200$, the kernel adapts tiling strategy to maintain SRAM residency, recovering performance. Across all sequence lengths tested, curves follow similar trajectories—demonstrating that the optimization strategy generalizes across the N dimension spanning production workloads.

Critically, all configurations maintain speedup ≥ 1.0 across the tested parameter space, with the majority achieving 1.2-2.0 $\times$ improvements. The absence of performance regressions validates KernelEvolve’s shape-aware optimization approach: generated kernels exploit fusion opportunities when tile configurations permit on-chip execution, while avoiding pathological cases through adaptive tiling strategies discovered during search.

Discussion. The PFFN case study demonstrates KernelEvolve’s ability to discover non-obvious fusion opportunities through systematic search over operator compositions and tiling configurations. While human experts might identify the matrix multiplication and normalization fusion conceptually, determining the precise tile dimensions and reuse strategies that maximize SRAM occupancy across production shape distributions requires extensive trial-and-error—effort KernelEvolve automates through graph-based search with execution feedback. The 1.5-2 $\times$ speedups achieved on production workloads validate that automated synthesis can match expert-level kernel implementations while reducing development cycles from weeks to hours.

5.4 Data Preprocessing Kernels on MTIA

MTIA is a custom silicon platform designed for both LLM and recommendation workloads. Unlike mature GPU ecosystems, MTIA presents unique kernel development challenges: not all PyTorch ATen operators are natively supported across hardware variants, and achieving optimal performance requires MTIA-specific tuning.

MTIA Kernel Coverage Challenge Running PyTorch models on MTIA requires kernel implementations for all ATen operators in the model graph. Missing kernels force either model rewrites or fallback to CPU execution,

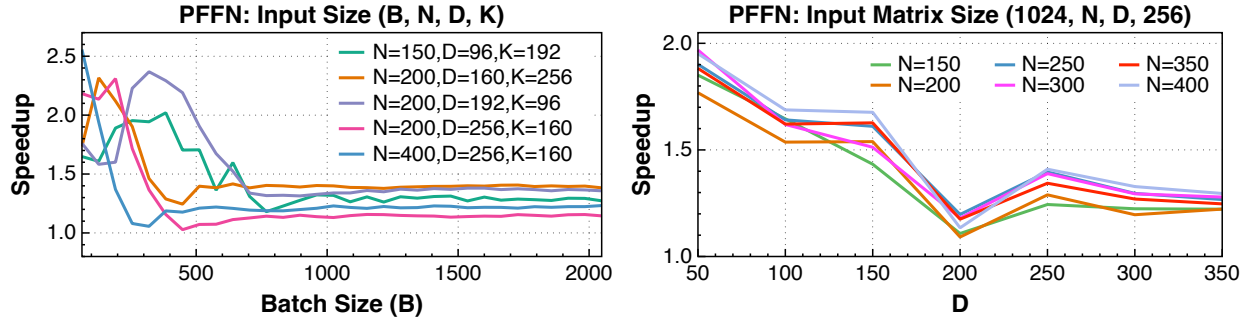


Figure 15 PFFN speedup on production shapes. Left: Speedup as a function of batch size B for five production (N, D, K) configurations, showing peak performance of 2.0-2.4 $\times$ at small batch sizes converging to 1.2-1.4 $\times$ at large batches as kernel launch overhead amortization reduces fusion advantages. Right: Speedup variation with input dimension D at fixed $B = 1024$, $K = 256$ across sequence lengths $N \in [150, 400]$, exhibiting non-monotonic behavior arising from tile size and SRAM capacity interactions.

both unacceptable for production latency requirements. Table 5 summarizes unsupported operators for two data preprocessing kernels across MTIA hardware generations.

AI Hardware	Data Preproc Operator	Missing ATen Ops
MTIA v2i	MapId MBDT	clamp.out, gather.out, sort.values_stable, all.all_out, _unique2 all.all_out, unique_consecutive
MTIA v3	MapId MBDT	clamp.out, sort.values_stable, _unique2 unique_consecutive

Table 5 Unsupported ATen operators on MTIA hardware for data preprocessing operators (MapId: MapIdTransform, MBDT: MergeBucketizedDenseTransform).

KernelEvolve addresses both model enablement and kernel optimization. On hardware with limited coverage (MTIA v2i), KernelEvolve-generated kernels provide the missing implementations required for model execution. On hardware with higher coverage (MTIA v3), KernelEvolve delivers performance gains through operator fusion and MTIA-specific tuning. The following sections evaluate two preprocessing kernels: MapIdTransform and MergeBucketizedDenseTransform (MBDT).

5.4.1 MapId Transform

MapIdTransform remaps sparse, high-cardinality categorical IDs to dense consecutive integers for embedding lookup. Given a sorted mapping tensor M containing known IDs, the kernel maps each input value to its 1-indexed position in M , reserving index 0 for unknown values.

Algorithm. For input tensor V and sorted mapping M :

1. Binary search to find insertion index: $\text{idx}_j = \text{bucketize}(v_j, M)$
2. Clamp index to valid range: $\text{idx}_j = \min(\text{idx}_j, |M| - 1)$
3. Validate match: if $M[\text{idx}_j] = v_j$, output $\text{idx}_j + 1$; else output 0

Input	values = [100, 300, 500, 200, 999] mapping = [100, 200, 300, 400, 500]
Output	[1, 3, 5, 2, 0]

Table 6 MapIdTransform example. Value 100 is at position 0 in mapping, outputs 1 (1-indexed). Value 300 is at position 2, outputs 3. Value 999 is not in mapping, outputs 0 (unknown).

The PyTorch reference implementation uses `torch.bucketize`, `torch.clamp`, `torch.gather`, and

`torch.where`—operators partially unsupported on MTIA v2i (Table 5).

```
class MapId(nn.Module):
    def forward(
        self,
        values: torch.Tensor,
    ) -> torch.Tensor:
        mapped_to_index = torch.clamp(
            torch.bucketize(values, self.mapping), max=self.mapping.numel() - 1
        )
        mapped_as_values = torch.gather(self.mapping, 0, mapped_to_index)
        mapped_values = torch.where(
            torch.eq(mapped_as_values, values), mapped_to_index + 1, 0
        )
        return mapped_values
```

Generated Kernel Optimizations. KernelEvolve synthesizes a fused Triton kernel that consolidates four PyTorch operators into a single accelerator invocation, applying three MTIA-targeted optimizations:

(1) *Operator Fusion.* The generated kernel fuses `bucketize`, `clamp`, `gather`, and `where` into a single kernel launch, eliminating three intermediate tensor materializations. Each thread block loads input values once, performs in-register binary search, validates matches, and writes final outputs—reducing global memory traffic by $4\times$ compared to operator-by-operator execution.

(2) *Compile-Time Loop Unrolling.* The binary search employs a fixed iteration bound of 20 steps, supporting mapping tables up to 2^{20} entries. This compile-time constant (`for _ in range(20)`) enables aggressive loop unrolling by the Triton compiler, converting control flow into predicated straight-line code. Search bounds are maintained in registers using vectorized `tl.where` operations for branchless conditional updates:

```
left = tl.where(search_active & (values > mapping_val), mid + 1, left)
right = tl.where(search_active & (values <= mapping_val), mid, right)
```

(3) *Coalesced Block-Parallel Execution.* The kernel organizes work into contiguous blocks via `BLOCK_SIZE` (a `tl.constexpr` parameter), where each program instance computes offsets as `block_start + tl.arange(0, BLOCK_SIZE)`. This layout ensures that adjacent threads access adjacent memory addresses, maximizing DRAM burst efficiency. Boundary conditions are handled through predicated loads and stores (`mask = offsets < n_elements`), avoiding divergent control flow while maintaining full memory coalescing across the warp.

Performance Analysis. Table 7 compares KernelEvolve-generated kernels against PyTorch on MTIA v2i and v3. As shown in Table 5, several ATen operators required by `MapIdTransform` lack native MTIA support, forcing PyTorch to execute CPU fallbacks with expensive host-device synchronization. KernelEvolve synthesizes a fused Triton kernel executing entirely on-device, providing both functional enablement and performance optimization.

On MTIA v2i, KernelEvolve achieves $3.28\text{--}3.48\times$ speedup for large batches (batch size = 10000), with consistent performance across mapping table sizes. Performance scales with batch size: $0.78\times$ at 2000 (launch overhead dominates), increasing to $1.38\times$ at 4000, $2.00\times$ at 6000, and $3.23\times$ at 10000, reaching peak $4.07\times$ at batch size 50000. This scaling reflects the trade-off between fixed kernel launch overhead and batch-dependent computation benefits.

On MTIA v3, latencies are substantially lower (0.035–0.174ms vs. 0.399–8.090ms on v1), reflecting improved hardware capabilities. Speedups are more modest at $1.05\text{--}1.36\times$, peaking at batch size 8000. The reduced gains arise from stronger PyTorch baselines on v2i due to improved native operator coverage and memory subsystem performance.

AI Hardware	Tensor Shape: (UniqueIDs $\times$ Batch)	PyTorch (ms)	Triton Kernel (ms)	Speedup
MTIA v2i	100 $\times$ 10000	1.623	0.466	3.48 $\times$
	500 $\times$ 10000	1.636	0.472	3.47 $\times$
	1000 $\times$ 10000	1.641	0.480	3.42 $\times$
	5000 $\times$ 10000	1.667	0.508	3.28 $\times$
	10000 $\times$ 2000	0.399	0.514	0.78 $\times$
	10000 $\times$ 4000	0.720	0.521	1.38 $\times$
	10000 $\times$ 6000	1.046	0.523	2.00 $\times$
	10000 $\times$ 8000	1.367	0.520	2.63 $\times$
	10000 $\times$ 10000	1.688	0.523	3.23 $\times$
	10000 $\times$ 50000	8.090	1.989	4.07 $\times$
	100 $\times$ 10000	0.061	0.058	1.05 $\times$
	500 $\times$ 10000	0.063	0.055	1.15 $\times$
MTIA v3	1000 $\times$ 10000	0.063	0.050	1.26 $\times$
	5000 $\times$ 10000	0.060	0.048	1.25 $\times$
	10000 $\times$ 2000	0.039	0.035	1.11 $\times$
	10000 $\times$ 4000	0.046	0.036	1.28 $\times$
	10000 $\times$ 6000	0.049	0.037	1.32 $\times$
	10000 $\times$ 8000	0.053	0.039	1.36 $\times$
	10000 $\times$ 10000	0.063	0.048	1.31 $\times$
	10000 $\times$ 50000	0.140	0.174	0.80 $\times$

MTIA v2i / v3 : Triton Kernel. Color: speedup , regression .

Table 7 MapIdTransform kernel performance: KernelEvolve-generated Triton kernels vs. PyTorch baseline on MTIA v2i and v3.

Across both hardware generations, KernelEvolve delivers robust speedups on large-batch workloads representative of production inference (up to 4.07 $\times$ on v2i, 1.36 $\times$ on v3). For edge cases where regressions occur on v2i and v3, runtime dispatch based on input dimensions ensures fallback to PyTorch, preventing performance degradation in deployment.

5.4.2 MergeBucketizedDense Transform

MBDT is a data preprocessing kernel that maps continuous features to discrete bin indices for embedding lookup in recommendation models. Given an input tensor and per-feature border lists, MBDT performs batched bucketization—a vectorized binary search assigning each value to its corresponding bin.

Operation. For input tensor $X \in \mathbb{R}^{F \times B}$ (F features, B batch size) and border lists $\{B_f\}_{f=1}^F$ where each $B_f = [b_1, b_2, \dots, b_{K_f}]$ is sorted, the output $Y_{f,i}$ is:

$$Y_{f,i} = \min \{k \mid X_{f,i} < B_f[k]\} \quad (2)$$

Example. Figure 16 illustrates MBDT execution with 2 features $\times$ 3 batch elements. Feature 0 has borders $[0.3, 0.6]$ creating 3 bins; Feature 1 has borders $[0.4, 0.7]$ creating 3 bins. The preprocessing stage flattens all borders into a single array with `inf` sentinels marking boundaries, enabling $O(1)$ lookup of each feature’s border range via offsets.

During parallel execution, each feature’s values undergo binary search against their respective borders. For Feature 0, input $[0.1, 0.4, 0.8]$ maps to bins $[0, 1, 2]$: value 0.1 falls in bin 0 ($x \leq 0.3$), value 0.4 in bin 1 ($0.3 < x \leq 0.6$), and value 0.8 in bin 2 ($x > 0.6$). Feature 1’s outputs are offset by 3 to ensure globally unique bin indices, yielding $[3, 4, 5]$ for inputs $[0.2, 0.5, 0.9]$. The final output tensor preserves the input shape with bin indices replacing continuous values.

Implementation. The core operation is a binary search over the borders array for each input value. The following pseudocode illustrates the sequential logic (the actual PyTorch baseline uses `torch.bucketize`):

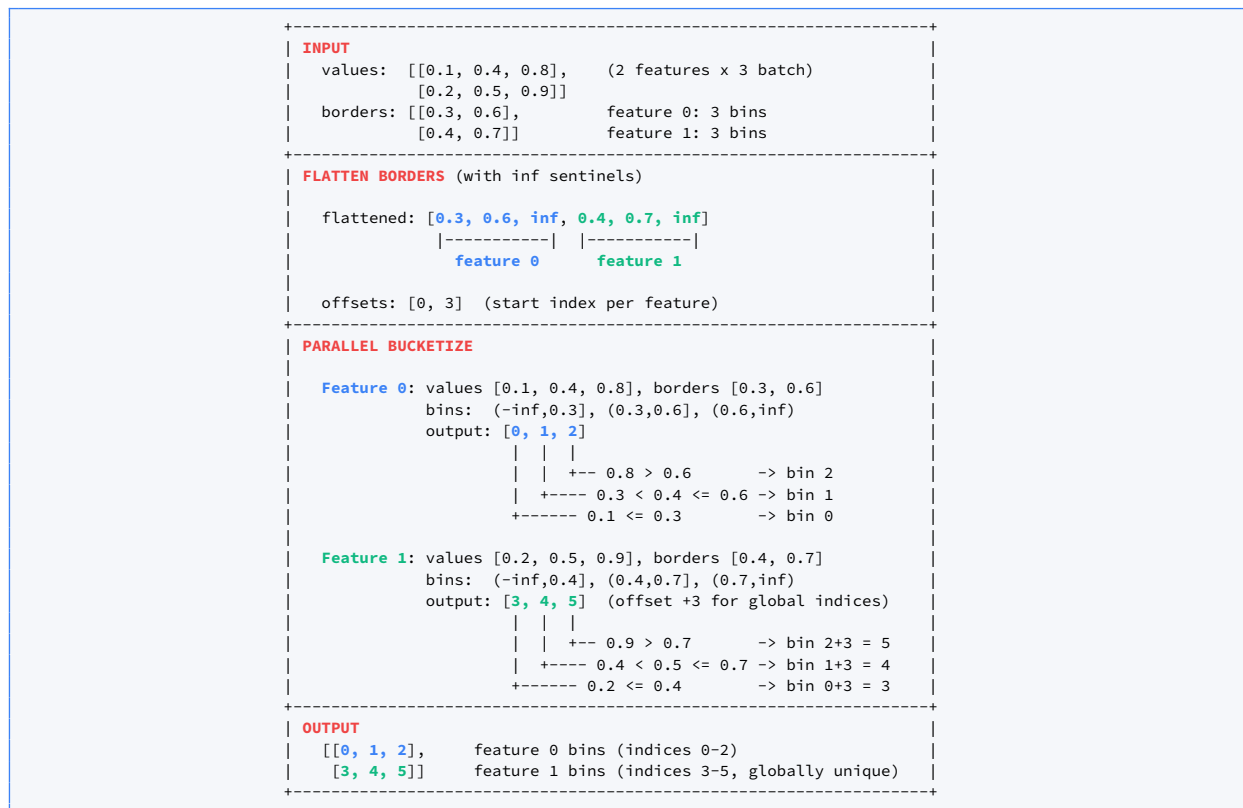


Figure 16 MBDT execution example. Section headers in red. Feature 0 data in blue. Feature 1 data in green. Borders are flattened with `inf` sentinels; output indices are offset per feature for global uniqueness.

```
def mbdt_sequential(values, borders_list, offsets):
    # values: [F, B], borders_list: List[Tensor], offsets: [F]
    output = torch.empty_like(values, dtype=torch.int64)
    for f in range(num_features):
        for i in range(batch_size):
            # Binary search for bucket index
            idx = torch.bucketize(values[f, i], borders_list[f])
            output[f, i] = idx + offsets[f]
    return output
```

The offset parameter assigns globally unique bin indices across features (e.g., Feature 1’s bins start at index 3 in the example above). The nested loops over features and batch elements present clear parallelization opportunities.

Performance Analysis. Figure 17 compares KernelEvolve-generated kernels against PyTorch (with `torch.compile`) across input configurations on MTIA v2i and v3. Configuration format is Batch × Features × Borders.

On MTIA v2i, KernelEvolve achieves substantial speedups ranging from 2.94× to 9.25×. Speedup scales with input size: smaller configurations ($64 \times 2 \times 2$) achieve 3.19×, while larger configurations ($2048 \times 2 \times 4$) reach 9.25×. This scaling behavior reflects the kernel’s ability to better amortize launch overhead and exploit parallelism as workload size increases.

On MTIA v3, absolute latencies are significantly lower (0.029–0.045ms vs. 0.027–0.064ms on v1), reflecting the next-generation hardware’s improved compute throughput. KernelEvolve achieves consistent speedups

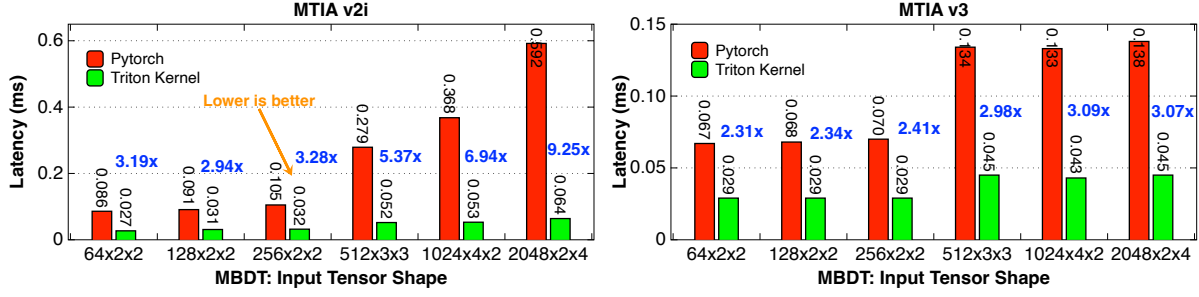


Figure 17 MBDT kernel latency comparison. Configuration format: Batch $\times$ Features $\times$ Borders. KernelEvolve achieves 2.94–9.25 $\times$ speedup on v2i and 2.31–3.09 $\times$ on v3, with larger speedups at higher batch sizes.

of 2.31–3.09 $\times$ across all configurations. The lower speedup magnitude compared to v1 is expected: v2i has higher native operator coverage (Table 5), resulting in a stronger PyTorch baseline with less room for optimization. Nevertheless, kernel fusion and vectorized execution still deliver meaningful gains.

Generated Kernel Optimizations. KernelEvolve generates a fused Triton kernel with several MTIA-specific optimizations:

Kernel Fusion. The entire bucketization pipeline—border lookup, binary search, and offset computation—executes in a single kernel launch, eliminating inter-kernel communication.

Vectorized Counting. Instead of scalar binary search, the kernel uses SIMD-vectorized counting: `values > border_val` is applied to blocks of 64–256 elements simultaneously. For typical small border arrays (3–10 elements), this $O(n)$ approach outperforms $O(\log n)$ binary search due to reduced control flow overhead and branch-free execution.

Adaptive Block Sizing. Block size is tuned based on input dimensions (64 for small, 128 for medium, 256 for large inputs) to maximize Processing Element utilization and ensure hardware saturation.

Register-Resident Computation. Intermediate results (left/right counts, averages) remain in registers throughout computation. No intermediate tensor allocations occur; results write directly to the output buffer.

MTIA-Specific Patterns. The kernel avoids constructs that fail MTIA compilation (e.g., `tl.where` in loops), using direct boolean-to-int conversion instead. Memory loads are coalesced with proper masking, and small border arrays are cached across processing blocks.

5.4.3 Summary

The MapIdTransform and MBMT evaluations demonstrate KernelEvolve’s dual value proposition on emerging hardware platforms: *enablement* and *optimization*. On MTIA v2i, where native operator coverage is limited, KernelEvolve-generated kernels provide the only viable on-device execution path—without them, PyTorch falls back to CPU for unsupported operators, incurring order-of-magnitude latency penalties. On MTIA v3, where coverage is more complete, KernelEvolve still delivers 2–3 $\times$ speedups through kernel fusion and hardware-specific tuning.

These results highlight a key insight: as new accelerators emerge, the gap between hardware availability and software ecosystem maturity creates a critical need for automated kernel generation. Traditional approaches—waiting for vendor libraries or hand-tuning by kernel experts—cannot scale to the diversity of operators and hardware variants in production. KernelEvolve addresses this by generating correct, optimized kernels from high-level operator specifications, reducing the time from hardware deployment to production readiness from months to hours.

Looking forward, we envision KernelEvolve as a foundational tool for heterogeneous accelerator ecosystems. As Meta continues to deploy next-generation MTIA hardware alongside NVIDIA and AMD GPUs, the ability to rapidly generate and optimize kernels across platforms becomes increasingly critical.

	Feature 0 (ad format)	Feature 1 (page id)
Input (3 users with [3, 4, 1] events)		
outer_len	[3 , 4 , 1]	[3 , 4 , 1]
inner_len	[1, 0, 2 , 0, 3, 1, 1 , 1]	[1, 1, 1 , 1, 1, 1, 1 , 1]
values	[1, 2, 3 , 4, 5, 6, 7, 8 , 9]	[1, 2, 3 , 4, 5, 6, 7 , 8]
Output (truncate to $N=2$ events)		
outer_len	[2 , 2 , 1]	[2 , 2 , 1]
inner_len	[1, 0 , 0, 3 , 1]	[1, 1 , 1, 1 , 1]
values	[1 , 4, 5, 6 , 9]	[1, 2 , 4, 5 , 8]

Figure 18 Batch Event Truncate with multiple features. Colors indicate users (User 0 , User 1 , User 2). Nested red highlights data discarded when truncating to $N=2$ events—showing which user’s data is being removed.

5.5 Sequence Learning: Batch Event Truncate

Event-based features (EBF) [Reddy et al. 2024] are a time-ordered sequence of interactions of a specific type, such as `ad_impression`, where each interaction contains multiple feature values (e.g., ad display format types, page id). EBF encodes user behavior sequences in ads ranking models as nested jagged tensors. Each batch contains three tensors: `outer_lengths` specifies the number of events per user, `inner_lengths` specifies the number of attributes per event, and `values` stores the flattened attribute data.

Operation. Figure 18 illustrates the Batch Event Truncate operation with multiple features. The input batch contains three users with [3, 4, 1] events respectively, and two features per event: Feature 0 (e.g., ad display format) with variable-length attributes, and Feature 1 (e.g., page id) with uniform single attributes. For Feature 0, User 0 has events with [1, 0, 2] attributes containing values [1, 2, 3]; User 1 has events with [0, 3, 1, 1] attributes containing values [4, 5, 6, 7, 8]; User 2 has a single event with 1 attribute containing value [9]. For Feature 1, all events have single attributes, with values [1, 2, 3] for User 0, [4, 5, 6, 7] for User 1, and [8] for User 2. When truncating to $N=2$ events, the operator retains only the first two events per user across all features simultaneously. For User 0, the third event is discarded—removing 2 attributes (values [2, 3]) from Feature 0 and 1 attribute (value [3]) from Feature 1. For User 1, the third and fourth events are removed—discarding attributes [1, 1] (values [7, 8]) from Feature 0 and attributes [1, 1] (values [6, 7]) from Feature 1. User 2 is unchanged since it has fewer than N events.

The operation requires coordinated index arithmetic across three nested levels and multiple features—a pattern poorly suited to standard tensor primitives. In production, sequences can reach 200 events with 5–32 features, making efficient batched truncation critical for serving latency and motivating a custom Triton kernel that processes all features in a single launch.

The original PyTorch implementation processes each feature event sequence independently in a loop—no batched variant existed due to the complexity of coordinating index arithmetic across nested jagged tensors. KernelEvolve automatically generates a batched Triton kernel that processes multiple features in parallel, a non-trivial optimization that would require significant manual engineering effort.

Performance Analysis. Table 8 compares the KernelEvolve-generated batched Triton kernel against the non-batched PyTorch baseline. The PyTorch implementation processes each feature sequentially, while the Triton kernel batches all features into a single launch.

Two factors drive the performance difference. First, when no truncation is needed ($\text{Max } N \geq \text{actual event count}$), PyTorch loops through each batch element comparing lengths individually, while the batched kernel performs a single vectorized comparison—yielding $9.8\times$ and $14.5\times$ speedups at higher feature counts. Second, when truncation is required, the batched kernel uses constant kernel launches for parallel processing versus PyTorch’s sequential iteration, achieving $1.4\text{--}2.0\times$ speedups.

Description	Feature Counts	Events	Max Length	PyTorch (ms)	Triton (ms)	Speedup
Single feature	1	200	100	0.148	0.313	1.0×
Single feature	1	200	200	0.148	0.109	1.4×
Prod multi-feature	5	200	100	0.788	0.571	1.4×
Prod multi-feature	9	200	200	1.443	0.148	9.8×
Large feature count	32	200	100	5.078	2.548	2.0×
Large feature count	32	200	200	5.085	0.350	14.5×

Table 8 Batch Event Truncate performance: KernelEvolve-generated batched Triton kernel vs. non-batched PyTorch baseline.

In production end-to-end benchmarks, the batched kernel achieves $2\times$ speedup over the PyTorch implementation. Notably, speedup scales with batch size, indicating that the batched kernel enables further model scaling by supporting additional event-based features without proportional latency increase. These results demonstrate that KernelEvolve can generate efficient batched implementations for operators where only sequential baselines exist, enabling significant latency reductions in production serving.

6 Future Directions

KernelEvolve demonstrates that LLM agents can generate production-quality kernels for heterogeneous accelerators, but this work represents only the first step toward a broader vision: *fully automated, heterogeneous hardware-aware code generation that scales across the entire AI infrastructure stack at Meta*.

Heterogeneous Hardware at Scale. As Meta’s AI infrastructure evolves to include next-generation MTIA, AMD MI series, ARM CPUs, and future NVIDIA architectures, the diversity of optimization targets will grow exponentially. We envision KernelEvolve as the unified kernel generation layer across this heterogeneous fleet—automatically adapting to new hardware through updated specifications rather than manual engineering. This requires developing hardware abstraction primitives that capture memory hierarchies, compute capabilities, and ISA constraints in a format amenable to LLM reasoning.

From Operators to Models. Current optimizations target individual operators and small modules, but the greatest performance gains lie at model level. Future work will extend KernelEvolve to reason about cross-layer fusion, global memory allocation, and end-to-end computation graphs. Combined with model transformation techniques—quantization, sparsity, architecture search—this enables co-optimization of model structure and kernel implementation, potentially discovering novel operator compositions that neither humans nor traditional compilers would identify.

Deeper Code Generation. Triton provides a productive abstraction, but certain optimizations require lower-level control. Extending KernelEvolve to modify MLIR dialects, direct PTX/SASS, or hardware diagnostic routines would unlock performance-critical scenarios where Triton’s abstractions become limiting. This vertical integration—from high-level DSLs to bare-metal code—positions LLM agents as general-purpose compilers rather than domain-specific tools.

Massively Parallel Search. The current tree search explores candidates sequentially, but kernel generation is inherently parallelizable. Relaxing consistency guarantees enables thousands of candidates to be evaluated simultaneously across distributed infrastructure, with eventual convergence to optimal solutions. This "infinite-width" search paradigm, combined with inference-time scaling laws, suggests that kernel quality may improve predictably with compute investment—a compelling property for production systems with strict performance targets.

Hardware-Specific Adaptation. Foundation models lack knowledge of proprietary accelerators like MTIA. Reinforcement learning from execution feedback—rewarding compilation success, correctness, and latency—offers a path to adapt generic models to specialized hardware without exposing proprietary details. This approach could enable rapid onboarding of new accelerators: deploy hardware, collect execution traces,

fine-tune the agent, and generate optimized kernels within days rather than months.

End-to-End Production Integration. The ultimate goal is seamless integration from model definition to production deployment. KernelEvolve should interface with AOT Inductor, model serving infrastructure, and continuous integration pipelines—automatically generating, validating, and deploying kernels as models and hardware evolve. This closed-loop system would reduce the human effort required to maintain performance across Meta’s rapidly evolving AI stack.

Sustainable AI Infrastructure. As LLM-based generation scales, resource efficiency becomes critical. Future work will quantify token consumption per kernel, optimize prompt design for minimal inference cost, and track carbon footprint across the search process [Wang et al. 2025a]. These metrics enable principled trade-offs between optimization depth and environmental impact, aligning KernelEvolve with broader sustainability goals in AI infrastructure [Wu et al. 2022].

7 Related Work

Kernel Optimization. High-performance kernel development has traditionally relied on vendor-optimized libraries (cuBLAS, cuDNN, rocBLAS) and auto-tuning frameworks. Halide decouples algorithm specification from scheduling, enabling portable optimization across hardware targets [Ragan-Kelley et al. 2013]. TVM extends this with learned cost models for automated schedule search [Chen et al. 2018]. Triton provides a Python-embedded DSL that abstracts GPU programming at the block level while exposing performance-critical tiling decisions [Tillet et al. 2019]. Recent work introduces higher-level and lower-level abstractions: NVIDIA’s CuTe DSL [NVIDIA 2025] as part of CUTLASS v4 provides composable layout and tensor abstractions for Tensor Core programming with Python JIT compilation. Meta’s TLX (Triton Low-Level Extensions) [Meta 2025b] adds warp-aware intrinsics and explicit pipeline control for NVIDIA GPU Hopper and Blackwell architectures. OpenAI’s Glue dialect [OpenAI 2025] exposes lower-level layout encoding within the Triton compiler stack whereas TileLang [Wang et al. 2025c] offers a composable tiled programming model with automatic layout inference across NVIDIA and AMD GPUs and Helion [Ansel 2025] compiles high-level PyTorch-like syntax to autotuned Triton code. These abstractions reduce development effort but still require substantial domain expertise for novel kernel transformations and struggle to generalize across heterogeneous hardware without manual adaptation.

LLM-Based Code and Kernel Generation. Beyond traditional kernel optimization, earlier work have applied evolutionary computations to improve general-purpose GPU code, such as machine learning kernels, by designing code transformation operators at the LLVM / MLIR level [Liou et al. 2020a, 2019, 2022, 2020b]. And, more recently, large language models (LLM) have demonstrated remarkable capabilities in general-purpose code synthesis [Chen 2021, Li et al. 2022, Roziere et al. 2023, team et al. 2025]. Recent work extends these capabilities to performance-critical domains: AlphaCode [Li et al. 2022, Team 2023] achieves competitive programming performance through large-scale sampling, while CodeRL [Le et al. 2022] incorporates execution feedback via reinforcement learning. These advances establish the foundation for applying LLMs to kernel development, where correctness and performance speedup are both essential.

Inference Time Scaling. Several recent systems start to apply LLMs specifically for GPU kernel synthesis. KernelBench [Ouyang et al. 2025] benchmarks LLM capabilities across operator, fusion, and model-level difficulty tiers. AutoTriton [Li et al. 2025] applies RL to Triton programming and KernelLLM [Fisches et al. 2025] explored supervised baseline for Triton kernel generation from PyTorch modules, while TritonRL [Woo et al. 2025] trains models with execution-guided rewards. KernelAgent [PyTorch 2025b] was designed around KernelLLM to generate verified Triton kernels from PyTorch programs based on a multi-agent system. GEAK [Wang et al. 2025b] targets AMD MI300X through agentic workflows, and Kevin [Baronio et al. 2025] employs multi-turn RL for CUDA generation. More recently, TritonX has been introduced as an agentic AI system that aims to generate functionally correct Triton kernels from PyTorch ATen operators [Hammond et al. 2025] – an important first step to enable automatic kernel generation for production deployed models to run on MTIAs. AlphaEvolve [Novikov et al. 2025] leverages evolutionary search with LLMs to optimize select stages of TPU/GPU kernels. While these systems demonstrate competitive results on isolated benchmarks, they target single hardware platforms with synthetic workloads, lacking heterogeneous hardware support, production operator coverage, and deployment infrastructure integration required for industry-scale adoption.

Finally, recent work demonstrates that model performance improves predictably with increased test-time compute [Snell et al. 2024]. Chain-of-thought prompting [Wei et al. 2022], tree-of-thought search [Yao et al. 2023], and self-consistency decoding [Wang et al. 2022] can enable complex reasoning through multi-path exploration. OpenAI’s o1 and DeepSeek-R1 [Guo et al. 2025] demonstrate substantial gains on mathematical and coding benchmarks through scaled inference. KernelEvolve builds on these insights, applying a tree-based search algorithm with execution feedback to systematically explore kernel optimization spaces for a wide collection of heterogeneous AI hardware. Other search algorithms, such as, [Toledo et al. 2025a], can be easily integrated to further improve KernelEvolve’s design space exploration and performance optimization results.

Overall, KernelEvolve differs from key prior work along the following three axes:

- *First*, it targets *heterogeneous hardware at scale* — generating optimized kernels for NVIDIA GPUs, AMD GPUs, and Meta’s custom MTIA accelerators from unified operator specifications. MTIA presents a unique challenge: as a proprietary architecture absent from public training corpora, effective kernel generation requires systematic knowledge injection of hardware constraints and programming idioms (Section 3.2.3).
- *Second*, KernelEvolve addresses *production operator diversity* beyond canonical benchmarks. Ads ranking models employ 200+ preprocessing operators with irregular access patterns and data-dependent control flow — operators that determine deployment architecture rather than merely affecting performance (Section 1).
- *Third*, it provides *deployment-integrated optimization* with continuous validation, multi-level profiling (system, kernel, intra-kernel), and serving infrastructure compatibility, enabling safe production rollout.

To our knowledge, KernelEvolve is the first LLM-based kernel coding system deployed at-scale for business-critical recommendation model inference.

8 Conclusion

This paper presents KernelEvolve, an agentic kernel generation framework addressing the three-dimensional diversity challenge in large-scale AI infrastructure: hardware heterogeneity across vendors and generations, model architectural diversity spanning retrieval to ranking stages, and kernel diversity from preprocessing to compute-intensive primitives. By formulating kernel optimization as graph-based search with retrieval-augmented prompting and persistent knowledge bases, KernelEvolve achieves 100% correctness across 480 operator-platform configurations and delivers $1.25\text{-}17\times$ speedups on production workloads, while reducing development time from weeks to hours.

Beyond performance gains, KernelEvolve demonstrates a fundamental shift in how we approach heterogeneous accelerator deployment. Incomplete kernel coverage forces costly disaggregated architectures with 10-20ms network overhead; automated synthesis eliminates these architectural penalties, transforming kernel availability from a deployment blocker to an enabler. As AI infrastructure evolves toward increasingly diverse accelerator fleets—integrating custom silicon, next-generation GPUs, and specialized processors—the gap between hardware innovation and software ecosystem maturity widens. Manual kernel development cannot scale to this combinatorial explosion of operators, models, and platforms.

We envision a future where LLM agents serve as the universal compilation layer for heterogeneous AI systems, automatically adapting to new hardware through knowledge injection rather than manual porting. KernelEvolve represents a first step toward this vision: demonstrating that agents can generate production-quality kernels for proprietary accelerators absent from training corpora, achieve competitive performance through inference-time scaling, and operate continuously in mission-critical infrastructure serving more than hundreds of trillions of daily inferences. The insights from this deployment—multi-granularity profiling integration, automated validation frameworks, and architecture-aware optimization—establish design principles for AI-assisted systems software that we believe will prove essential as the hardware landscape continues to fragment and diversify. The path forward requires not just better kernels, but fundamentally new approaches to bridging the hardware-software gap at scale.

9 Contributors

Project Leads

- Gang Liao, Gaoxiang Liu

Core Contributors

- **Monetization Infra and Ranking:** Gang Liao, Hongsen Qin, Ying Wang, Yavuz Yetim, Jia Jiunn Ang, Xiayu Yu, Yihan He, Feng Shi, Zewei Jiang, Chunli Fu, Ruichao Xiao, Dianshi Li, Gaoxiang Liu
- **FAIR SysML:** Alicia Golden, Michael Kuchnik, Samuel Hsia, Carole-Jean Wu
- **MTIA Software:** Roman Levenstein, Kunming Ho, Haishan Zhu, Alec Hammond, Richard Li, Ajit Mathews, Kaustubh Gondkar

Contributors

- **Ad Use Cases:** Zhou Fang, Abdul Zainul-Abedin, Ketan Singh
- **Serverless Compute:** Sean Zhang, Noah Weller, Zach Marine, Wyatt Cook
- **AI Compilers & Runtimes:** Hongtao Yu, Wenyuan Chi, Barney Huang

Supervision

- Liyuan Li, Nathan Yan, Varna Puvvada, Uladzimir Pashkevich, Matt Steiner

10 Acknowledgments

Yoram Bachrach, Rick Chang, Yuanwei Fang, Jun Ge, Danilo Carvalho Grael, Karen Hambardzumyan, Nathan Hu, Keren Huang, Joe Isaacson, Martin Josifoski, Minjang Kim, Irene Liu, Alexey Loginov, Abhi Pandey, Srivatsan Ramesh, Praveen Ramachandran, Mark Saroufim, Dev (Devashish) Shankar, Rithin Shetty, Bidit Sharma, Jake Siso, Oleksandr Stashuk, Tejas Venkateswaran, Edan Toledo, Laura Wang, Shiguang Wang, Zhaodong Wang, Shawn Xu, Feixiong Zhang, Xudong Zhang, Mingjie Zhu

References

- Jason Ansel. Helion: Python-embedded domain-specific language (dsl) for authoring machine learning kernels. 2025. <https://github.com/pytorch/helion>.
- Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 929–947, 2024.
- Anthropic. Introducing the model context protocol. *Engineering at Anthropic*, 2024. <https://www.anthropic.com/news/model-context-protocol>.
- Anthropic. Effective context engineering for ai agents. *Engineering at Anthropic*, 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Andrew Audibert, Yang Chen, Dan Graur, Ana Klimovic, Jiří Šimša, and Chandramohan A Thekkath. tf. data service: A case for disaggregating ml input data processing. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*, pages 358–375, 2023.
- Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-turn rl for generating cuda kernels. *arXiv preprint arXiv:2507.11948*, 2025.
- Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, et al. Cwm: An open-weights llm for research on code generation with world models. *arXiv preprint arXiv:2510.02387*, 2025.
- Mark Chen. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, 2018.
- Joel Coburn, Chunqiang Tang, Sameer Abu Asal, Neeraj Agrawal, Raviteja Chinta, Harish Dixit, Brian Dodds, Saritha Dwarakapuram, Amin Firoozshahian, Cao Gao, et al. Meta’s second generation ai chip: Model-chip co-design and productionization experiences. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pages 1689–1702, 2025.
- Kenneth De Jong. Evolutionary computation: a unified approach. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 373–388, 2017.
- DeepSeek. Deepgemma. *github*, 2025. <https://github.com/deepseek-ai/DeepGEMM>.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407, 2024.
- Amin Firoozshahian, Joel Coburn, Roman Levenstein, Rakesh Nattoji, Ashwin Kamath, Olivia Wu, Gurdeepak Grewal, Harish Aepala, Bhasker Jakka, Bob Dreyer, et al. Mtia: First generation silicon targeting meta’s recommendation systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–13, 2023.
- Zacharias V. Fisches, Sahan Paliskara, Simon Guo, Alex Zhang, Joe Spisak, Chris Cummins, Hugh Leather, Gabriel Synnaeve, Joe Isaacson, Aram Markosyan, and Mark Saroufim. Kernelllm: Making kernel development more accessible, 6 2025. <https://huggingface.co/facebook/KernelLLM>. Corresponding authors: Aram Markosyan, Mark Saroufim.
- Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. Rlef: Grounding code llms in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089*, 2024.
- Boris Grubic, Yang Wang, Tyler Petrochko, Ran Yaniv, Brad Jones, David Callies, Matt Clarke-Lauer, Dan Kelley, Soteris Demetriou, Kenny Yu, et al. Conveyor: {One-Tool-Fits-All} continuous software deployment at meta. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 325–342, 2023.
- Yue Guan, Yuanwei Fang, Keren Zhou, Corbin Robeck, Manman Ren, Zhongkai Yu, Yufei Ding, and Adnan Aziz. Kperfir: towards an open and compiler-centric ecosystem for gpu kernel performance tooling on modern ai workloads. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation, OSDI ’25, USA*, 2025. USENIX Association. ISBN 978-1-939133-47-2.

- Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- Alec M. Hammond, Aram Markosyan, Aman Dontula, Simon Mahns, Zacharias Fisches, Dmitrii Pedchenko, Keyur Muzumdar, Natacha Supper, Mark Saroufim, Joe Isaacson, Laura Wang, Warren Hunt, Kaustubh Gondkar, Roman Levenstein, Gabriel Synnaeve, Richard Li, Jacob Kahn, and Ajit Mathews. Agentic operator generation for ml asics, 2025. <https://arxiv.org/abs/2512.10977>.
- Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.
- Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.
- Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. Mlir: A compiler infrastructure for the end of moore’s law. *arXiv preprint arXiv:2002.11054*, 2020.
- Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35:21314–21328, 2022.
- Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, et al. Autotriton: Automatic triton programming with reinforcement learning in llms. *arXiv preprint arXiv:2507.05687*, 2025.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.
- Gang Liao and Daniel J Abadi. Filescale: Fast and elastic metadata management for distributed file systems. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*, pages 459–474, 2023.
- Gang Liao, Amol Deshpande, and Daniel J Abadi. Flock: A low-cost streaming query engine on faas platforms. *arXiv preprint arXiv:2312.16735*, 2023.
- Gang Liao, Ye Liu, Jianjun Chen, and Daniel J Abadi. Bullion: A column store for machine learning. *Proceedings of 15th Conference on Innovative Data Systems Research (CIDR)*, 2025.
- Gang Liao, Yujia He, Abdullah Ozturk, Zhouyang Li, Ying Wang, Zhitong Guo, Hongsen Qin, Yaobin Qin, Tao Yang, Zewei Jiang, Dianshi Li, Jort Gemmeke, Jiangyuan Li, Liyuan Li, Nathan Yan, Masha Basmanova, Uladzimir Pashkevich, Matt Steiner, Pedro Pedreira, Rob Fergus, Anirudh Goyal, Carole-Jean Wu, Gaoxiang Liu, Andrew Witten, and Daniel J. Abadi. Experience graphs: The data foundation for self-improving agents, 2026. <https://arxiv.org/abs/2606.29823>.
- Jhe-Yu Liou, Stephanie Forrest, and Carole-Jean Wu. Genetic improvement of gpu code. In *2019 IEEE/ACM International Workshop on Genetic Improvement (GI)*, 2019.
- Jhe-Yu Liou, Xiaodong Wang, Stephanie Forrest, and Carole-Jean Wu. Gevo-ml: a proposal for optimizing ml code with evolutionary computation. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion, GECCO ’20*, page 1849–1856, 2020a.
- Jhe-Yu Liou, Xiaodong Wang, Stephanie Forrest, and Carole-Jean Wu. Gevo: Gpu code optimization using evolutionary computation. *ACM Trans. Archit. Code Optim.*, 17(4), 2020b.
- Jhe-Yu Liou, Muaaz Awan, Steven Hofmeyr, Stephanie Forrest, and Carole-Jean Wu. Understanding the power of evolutionary computation for gpu code optimization. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*, 2022.
- Xinchen Luo, Jiangxia Cao, Tianyu Sun, Jinkai Yu, Rui Huang, Wei Yuan, Hezheng Lin, Yichen Zheng, Shiyao Wang, Qigen Hu, et al. Qarm: Quantitative alignment multi-modal recommendation at kuaishou. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 5915–5922, 2025.
- Simon Marlow and Pepe Iborra. Indexing code at scale with glean. *Engineering at Meta*, 2024. <https://engineering.fb.com/2024/12/19/developer-tools/glean-open-source-code-indexing/>.

Meta. Pytorch profiler. *Pytorch at Meta*, 2021. https://docs.pytorch.org/tutorials/recipes/recipes/profiler_recipe.html.

Meta. Our next-generation meta training and inference accelerator. *Meta AI blog*, 2024. <https://ai.meta.com/blog/next-generation-meta-training-inference-accelerator-AI-MTIA/>.

Meta. Tritonbench. *Pytorch at Meta*, 2025a. <https://github.com/meta-pytorch/tritonbench>.

Meta. Tlx - triton low-level language extensions. 2025b. <https://github.com/facebookexperimental/triton>.

Derek G. Murray, Jiří Šimša, Ana Klimovic, and Ihor Indyk. tf.data: a machine learning data processing framework. *Proc. VLDB Endow.*, 14(12):2945–2958, July 2021. ISSN 2150-8097. doi: 10.14778/3476311.3476374. <https://doi.org/10.14778/3476311.3476374>.

Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, et al. Mlgym: A new framework and benchmark for advancing ai research agents. *arXiv preprint arXiv:2502.14499*, 2025.

Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091*, 2019.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. <https://arxiv.org/abs/2506.13131>.

Nvidia. Nsight compute cli - v2025.3.1. *Nvidia Docs*, 2025. <https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html>.

NVIDIA. Introduction to cute dsl. 2025. https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/cute_dsl_general/dsl_introduction.html.

OpenAI. Introduction to gluon. 2025. <https://github.com/triton-lang/triton/blob/main/python/tutorials/gluon/01-intro.py>.

Anne Ouyang, Simon Guo, Simran Arora, Alex L Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517*, 2025.

Rachel Potvin and Josh Levenberg. Why google stores billions of lines of code in a single repository. *Communications of the ACM*, 59(7):78–87, 2016.

PyTorch. Fbgemm. *github*, 2025a. <https://github.com/pytorch/FBGEMM>.

PyTorch. Kernelagent — multi-agent gpu kernel synthesis, 2025b. <https://github.com/meta-pytorch/KernelAgent>.

Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Xuanhe Zhou, Yufei Huang, Chaojun Xiao, et al. Tool learning with foundation models. *ACM Computing Surveys*, 57(4):1–40, 2024.

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6):519–530, 2013.

Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems*, 36:10299–10315, 2023.

Sri Reddy, Habiya Beg, Arnold Overwijk, and Sean O’Byrne. Sequence learning: A paradigm shift for personalized ads recommendations. *Engineering at Meta*, 2024. <https://engineering.fb.com/2024/11/19/data-infrastructure/sequence-learning-personalized-ads-recommendations/>.

Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Alireza Sahraei, Soteris Demetriou, Amirali Sobhghol, Haoran Zhang, Abhigna Nagaraja, Neeraj Pathak, Girish Joshi, Carla Souza, Bo Huang, Wyatt Cook, et al. Xfaas: Hyperscale and low cost serverless functions at meta. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 231–246, 2023.

- Harshit Saokar, Soteris Demetriou, Nick Magerko, Max Kontorovich, Josh Kirstein, Margot Leibold, Dimitrios Skarlatos, Hitesh Khandelwal, and Chunqiang Tang. {ServiceRouter}: Hyperscale and minimal cost service mesh at meta. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 969–985, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.
- Chunqiang Tang, Kenny Yu, Kaushik Veeraraghavan, Jonathan Kaldor, Scott Michelson, Thawan Kooburat, Aravind Anbudurai, Matthew Clark, Kabir Gogia, Long Cheng, et al. Twine: A unified cluster management system for shared infrastructure. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 787–803, 2020.
- AlphaCode Team. Alphacode 2 technical report. 2023. https://storage.googleapis.com/deepmind-media/AlphaCode2/AlphaCode2_Tech_Report.pdf.
- FAIR CodeGen team, Jade Copet, Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, David Zhang, Kunhao Zheng, Jordi Armengol-Estapé, Pedram Bashiri, Maximilian Beck, Pierre Chambon, Abhishek Charnalia, Chris Cummins, Juliette Decugis, Zacharias V. Fisches, François Fleuret, Fabian Gloeckle, Alex Gu, Michael Hassid, Daniel Haziza, Badr Youbi Idrissi, Christian Keller, Rahul Kindi, Hugh Leather, Gallil Maimon, Aram Markosyan, Francisco Massa, Pierre-Emmanuel Mazaré, Vegard Mella, Naila Murray, Keyur Muzumdar, Peter O’Hearn, Matteo Pagliardini, Dmitrii Pedchenko, Tal Remez, Volker Seeker, Marco Selvi, Oren Sultan, Sida Wang, Luca Wehrstedt, Ori Yoran, Lingming Zhang, Taco Cohen, Yossi Adi, and Gabriel Synnaeve. Cwm: An open-weights llm for research on code generation with world models, 2025. <https://arxiv.org/abs/2510.02387>.
- Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.
- Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Maria Lupidi, Andrei Lupu, Roberta Raileanu, Kelvin Niu, Tatiana Shavrina, Jean-Christophe Gagnon-Audet, Michael Shvartsman, Shagun Sodhani, Alexander H. Miller, Abhishek Charnalia, Derek Dunfield, Carole-Jean Wu, Pontus Stenertorp, Nicola Cancedda, Jakob Nicolaus Foerster, and Yoram Bachrach. Ai research agents for machine learning: Search, exploration, and generalization in mle-bench, 2025a. <https://arxiv.org/abs/2507.02554>.
- Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Maria Lupidi, et al. Ai research agents for machine learning: Search, exploration, and generalization in mle-bench. *arXiv preprint arXiv:2507.02554*, 2025b.
- Oreste Villa, Mark Stephenson, David Nellans, and Stephen W Keckler. Nvbit: A dynamic binary instrumentation framework for nvidia gpus. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 372–383, 2019.
- Irene Wang, Newsha Ardalani, Mostafa Elhoushi, Daniel Jiang, Samuel Hsia, Ekin Sumbul, Divya Mahajan, Carole-Jean Wu, and Bilge Acun. Catransformers: Carbon aware transformers through joint model-hardware optimization, 2025a. <https://arxiv.org/abs/2505.01386>.
- Jianghui Wang, Vinay Joshi, Saptarshi Majumder, Xu Chao, Bin Ding, Ziqiong Liu, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. Geak: Introducing triton kernel ai agent & evaluation benchmarks. *arXiv preprint arXiv:2507.23194*, 2025b.
- Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, et al. Tilelang: A composable tiled programming model for ai systems, 2025. URL <https://arxiv.org/abs/2504.17577>, 2, 2025c.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837, 2022.
- Jiin Woo, Shaowei Zhu, Allen Nie, Zhen Jia, Yida Wang, and Youngsuk Park. Tritonrl: Training llms to think and code triton without cheating. *arXiv preprint arXiv:2510.17891*, 2025.
- Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga Behram, James Huang, Charles Bai, Michael Gschwind, Anurag Gupta, Myle Ott, Anastasia Melnikov, Salvatore Candido, David Brooks, Geeta Chauhan, Benjamin Lee, Hsien-Hsin S. Lee, Bugra Akyildiz, Maximilian Balandat, Joe Spisak, Ravi Jain, Mike Rabbat, and Kim Hazelwood. Sustainable ai: Environmental implications, challenges and opportunities, 2022. <https://arxiv.org/abs/2111.00364>.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Zhichen Zeng, Xiaolong Liu, Mengyue Hang, Xiaoyi Liu, Qinghai Zhou, Chaofer Yang, Yiqun Liu, Yichen Ruan, Laming Chen, Yuxin Chen, et al. Interformer: Effective heterogeneous interaction learning for click-through rate prediction. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 6225–6233, 2025.
- Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. Actions speak louder than words: trillion-parameter sequential transducers for generative recommendations. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*, 2024.
- Buyun Zhang, Liang Luo, Xi Liu, Jay Li, Zeliang Chen, Weilin Zhang, Xiaohan Wei, Yuchen Hao, Michael Tsang, Wenjun Wang, et al. Dhen: A deep and hierarchical ensemble network for large-scale click-through rate prediction. *arXiv preprint arXiv:2203.11014*, 2022.
- Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Daifeng Guo, Yanli Zhao, Shen Li, Yuchen Hao, Yantao Yao, et al. Wukong: Towards a scaling law for large-scale recommendation. *arXiv preprint arXiv:2403.02545*, 2024.
- Ning Zhang and George Wang. Unlock peak performance on amd gpus with triton kernel optimizations. *Engineering at AMD*, 2025. <https://rocm.blogs.amd.com/software-tools-optimization/kernel-development-optimizations-with-triton-on-/README.html>.
- Mark Zhao, Niket Agarwal, Aarti Basant, Buğra Gedik, Satadru Pan, Mustafa Ozdal, Rakesh Komuravelli, Jerry Pan, Tianshu Bao, Haowei Lu, et al. Understanding data storage and ingestion for large-scale deep recommendation model training: Industrial product. In *Proceedings of the 49th annual international symposium on computer architecture*, pages 1042–1057, 2022.
- Guorui Zhou, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Shiyao Wang, et al. Onerec technical report. *arXiv preprint arXiv:2506.13695*, 2025a.
- Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, et al. Onerec-v2 technical report. *arXiv preprint arXiv:2508.20900*, 2025b.
- Keren Zhou, Yuanwei Fang, and Corbin Robeck. Proton: Portable performance profiling. *Triton Conference’25*, 2025c. https://docs.google.com/presentation/d/1rBniQNaFUYJQE_6bZtyhNSSNZmE7l9tc.

Appendix

A 1D Convolution

Note: The candidate presented here was randomly selected from a pool of high-performing solutions rather than representing the single best-performing variant. Due to data confidentiality constraints, production data could not be used directly. Instead, we employ synthetic data that preserves the statistical properties (Min, Max, Mean, and STD) of the original production dataset. This synthetic dataset has been validated to yield equivalent speedup characteristics to those observed with production data. Additionally, the internal Triton trunk version used in production differs slightly from the open-source release.

```
"""
Auto-generated TritonBench Operator for conv1d

=====
GENERATION METADATA
=====
Generated by:    kernel_evolve CLI (TritonBench Operator generator)
Generated at:    2025-11-21 03:14:31
Operator name:   conv1d
Precision:       fp16
Device:          cuda
Output dir:      /tmp/kernel_evolve_eval
=====

This file was automatically generated by the kernel_evolve evals framework.
DO NOT EDIT THIS FILE MANUALLY - regenerate it using the CLI tool.
"""

import argparse
import sys
from typing import Any, Callable, Generator, List, Optional

# =====
# Kernel Evolve Generated Models
# =====

import torch
import torch.nn as nn
import triton
import triton.language as tl
from tritonbench.utils.triton_op import BenchmarkOperator, register_benchmark

def generate_synthetic_production_data(dtype=torch.float32, device="cuda"):
    """
    Generate synthetic data matching production data characteristics.

    Creates realistic test data with controlled statistical properties
    suitable for performance evaluation.

    Production data statistics (analyzed 2025-12-12):
    - Input: shape=(2048, 96, 200), dtype=torch.float32, mean=0.006413, std=0.041105
    - Weight: shape=(96, 96, 3), dtype=torch.float32, mean=-0.000024, std=0.047870
    - Bias: shape=(96,), dtype=torch.float32, mean=-0.003198, std=0.033662

    Args:
```

```

dtype: Data type for tensors (default: torch.float32, matching production data)
device: Device to place tensors on (default: "cuda")

Returns:
    Tuple of (input_tensor, weight_tensor, bias_tensor)
"""
torch.manual_seed(42)

# Production data shape parameters
max_batch_size = 2048
in_channels = 96
out_channels = 96
seq_length = 200
kernel_size = 3

input_full = (
    torch.randn(max_batch_size, in_channels, seq_length, device=device, dtype=dtype)
    * 0.041105
    + 0.006413
)

weight_tensor = (
    torch.randn(out_channels, in_channels, kernel_size, device=device, dtype=dtype)
    * 0.047870
    - 0.000024
)

bias_tensor = (
    torch.randn(out_channels, device=device, dtype=dtype) * 0.033662 - 0.003198
)

return input_full, weight_tensor, bias_tensor

# Global cache to avoid re-packing weights on every call
# Set DISABLE_WEIGHT_CACHE=True to measure "cold start" performance including weight packing
_WEIGHT_PACK_CACHE = {}
_DISABLE_WEIGHT_CACHE = True # Set to True to benchmark without caching

class PytorchModel(nn.Module):
    def __init__(self) -> None:
        super().__init__()

    def forward(self, input_tensor, weight_tensor, bias_tensor, use_conv2d=False):
        if use_conv2d:
            input_tensor = input_tensor.unsqueeze(2).to(
                memory_format=torch.channels_last
            )
            weight_tensor = weight_tensor.unsqueeze(2)
            return torch.nn.functional.conv2d(
                input_tensor,
                weight_tensor,
                bias_tensor,
                stride=(1, 1),
                padding=(0, 1),
                dilation=1,
            ).squeeze(2)
        else:
            return torch.nn.functional.conv1d(

```

```

        input_tensor,
        weight_tensor,
        bias_tensor,
        stride=1,
        padding=1,
        dilation=1,
        groups=1,
    )

# =====
# Autotune configs for conv kernel
# Expanded and tuned for M-heavy shapes and small K
# =====
autotune_configs = [
    # Small/latency-oriented
    triton.Config(
        {
            "BLOCK_M": 32,
            "BLOCK_N": 64,
            "BLOCK_K": 32,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=2,
        num_stages=3,
    ),
    triton.Config(
        {
            "BLOCK_M": 32,
            "BLOCK_N": 128,
            "BLOCK_K": 32,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=4,
        num_stages=2,
    ),
    triton.Config(
        {
            "BLOCK_M": 64,
            "BLOCK_N": 32,
            "BLOCK_K": 32,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=2,
        num_stages=4,
    ),
    # Balanced
    triton.Config(
        {
            "BLOCK_M": 64,
            "BLOCK_N": 64,
            "BLOCK_K": 32,
            "GROUP_M": 4,
            "WARP_SPECIALIZE": False,
        },
    ),

```

```

        num_warps=4,
        num_stages=2,
    ),
    triton.Config(
        {
            "BLOCK_M": 64,
            "BLOCK_N": 128,
            "BLOCK_K": 32,
            "GROUP_M": 4,
            "WARP_SPECIALIZE": False,
        },
        num_warps=8,
        num_stages=2,
    ),
    triton.Config(
        {
            "BLOCK_M": 128,
            "BLOCK_N": 64,
            "BLOCK_K": 32,
            "GROUP_M": 4,
            "WARP_SPECIALIZE": False,
        },
        num_warps=4,
        num_stages=3,
    ),
    triton.Config(
        {
            "BLOCK_M": 128,
            "BLOCK_N": 128,
            "BLOCK_K": 32,
            "GROUP_M": 4,
            "WARP_SPECIALIZE": False,
        },
        num_warps=8,
        num_stages=3,
    ),
    # Deeper K tiles
    triton.Config(
        {
            "BLOCK_M": 64,
            "BLOCK_N": 64,
            "BLOCK_K": 64,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=4,
        num_stages=3,
    ),
    triton.Config(
        {
            "BLOCK_M": 128,
            "BLOCK_N": 64,
            "BLOCK_K": 64,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=4,
        num_stages=4,
    ),

```



```

),
triton.Config(
    {
        "BLOCK_M": 64,
        "BLOCK_N": 128,
        "BLOCK_K": 64,
        "GROUP_M": 8,
        "WARP_SPECIALIZE": False,
    },
    num_warps=8,
    num_stages=3,
),
triton.Config(
    {
        "BLOCK_M": 128,
        "BLOCK_N": 128,
        "BLOCK_K": 64,
        "GROUP_M": 8,
        "WARP_SPECIALIZE": False,
    },
    num_warps=8,
    num_stages=4,
),
# M-heavy problems
triton.Config(
    {
        "BLOCK_M": 256,
        "BLOCK_N": 64,
        "BLOCK_K": 32,
        "GROUP_M": 8,
        "WARP_SPECIALIZE": False,
    },
    num_warps=8,
    num_stages=4,
),
triton.Config(
    {
        "BLOCK_M": 256,
        "BLOCK_N": 128,
        "BLOCK_K": 32,
        "GROUP_M": 8,
        "WARP_SPECIALIZE": False,
    },
    num_warps=8,
    num_stages=4,
),
triton.Config(
    {
        "BLOCK_M": 256,
        "BLOCK_N": 64,
        "BLOCK_K": 64,
        "GROUP_M": 8,
        "WARP_SPECIALIZE": False,
    },
    num_warps=8,
    num_stages=5,
),
triton.Config(

```

```

        {
            "BLOCK_M": 256,
            "BLOCK_N": 128,
            "BLOCK_K": 64,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=8,
        num_stages=5,
    ),
    # Aggressive N-tiling
    triton.Config(
        {
            "BLOCK_M": 128,
            "BLOCK_N": 256,
            "BLOCK_K": 32,
            "GROUP_M": 4,
            "WARP_SPECIALIZE": False,
        },
        num_warps=16,
        num_stages=3,
    ),
    triton.Config(
        {
            "BLOCK_M": 64,
            "BLOCK_N": 256,
            "BLOCK_K": 32,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=16,
        num_stages=3,
    ),
    # Larger K tiles to saturate tensor cores
    triton.Config(
        {
            "BLOCK_M": 128,
            "BLOCK_N": 128,
            "BLOCK_K": 128,
            "GROUP_M": 4,
            "WARP_SPECIALIZE": False,
        },
        num_warps=8,
        num_stages=4,
    ),
    triton.Config(
        {
            "BLOCK_M": 256,
            "BLOCK_N": 64,
            "BLOCK_K": 128,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=8,
        num_stages=5,
    ),
    # Very large M-tiles for huge sequences/batches
    triton.Config(

```

```

        {
            "BLOCK_M": 512,
            "BLOCK_N": 64,
            "BLOCK_K": 64,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=8,
        num_stages=5,
    ),
    triton.Config(
        {
            "BLOCK_M": 512,
            "BLOCK_N": 128,
            "BLOCK_K": 64,
            "GROUP_M": 8,
            "WARP_SPECIALIZE": False,
        },
        num_warps=16,
        num_stages=5,
    ),
]

# =====
# Weight packing kernel: [Cout, Cin_g, Ksz] -> [Cin_g*Ksz, Cout]
# Vectorized copy with 2D tiling. This improves global load/store efficiency.
# =====
@triton.jit

def pack_conv1d_weight_kernel(
    w_in_ptr,
    w_out_ptr,
    Cout,
    Cin_g,
    Ksz,
    BLOCK_R: tl.constexpr, # tile along rows (Cin_g*Ksz)
    BLOCK_C: tl.constexpr, # tile along cols (Cout)
):
    R = Cin_g * Ksz # rows of packed matrix
    C = Cout # cols of packed matrix
    pid_r = tl.program_id(0)
    pid_c = tl.program_id(1)

    offs_r = pid_r * BLOCK_R + tl.arange(0, BLOCK_R) # [BR]
    offs_c = pid_c * BLOCK_C + tl.arange(0, BLOCK_C) # [BC]
    mask_r = offs_r < R
    mask_c = offs_c < C

    # Treat input as A: [Cout, Cin_g*Ksz]; row-stride ldA = Cin_g*Ksz
    ldA = R
    a_ptrs = w_in_ptr + offs_c[:, None] * ldA + offs_r[None, :] # [BC, BR]
    a_mask = mask_c[:, None] & mask_r[None, :]
    # Cache in L2 to promote reuse across blocks that share Cout slices
    a = tl.load(a_ptrs, mask=a_mask, other=0.0, cache_modifier=".cg")

    # Output is B: [Cin_g*Ksz, Cout]; row-stride ldB = Cout
    ldB = C

```

```

b_ptrs = w_out_ptr + offs_r[None, :] * ldB + offs_c[:, None] # [BC, BR]
tl.store(b_ptrs, a, mask=a_mask)

# =====
# Conv1D GEMM-style kernel with packed weights (3D tiling grid)
# input: [B, Cin, Lin] contiguous
# weight_packed: [Cin_g*Ksz, Cout] contiguous
# output: [B, Cout, Lout]
# =====
@triton.autotune(
    configs=autotune_configs,
    key=["M", "N", "K", "stride", "dilation", "kernel_size", "Cin_g"],
)
@triton.jit
def conv1d_gemm_kernel(
    input_ptr,
    weight_packed_ptr,
    bias_ptr,
    output_ptr,
    # Dimensions (constexpr for specialization)
    B: tl.constexpr,
    Cin: tl.constexpr,
    Cout: tl.constexpr,
    Lin: tl.constexpr,
    Lout: tl.constexpr,
    G: tl.constexpr,
    # Matmul dims
    M: tl.constexpr, # M = B * Lout
    N: tl.constexpr, # N = out_channels_per_group
    K: tl.constexpr, # K = Cin_g * kernel_size
    Cin_g: tl.constexpr, # in_channels per group
    # Conv hyperparams
    kernel_size: tl.constexpr,
    stride: tl.constexpr,
    padding: tl.constexpr,
    dilation: tl.constexpr,
    has_bias: tl.constexpr,
    # Tunables (from autotuner)
    BLOCK_M: tl.constexpr,
    BLOCK_N: tl.constexpr,
    BLOCK_K: tl.constexpr,
    GROUP_M: tl.constexpr,
    WARP_SPECIALIZE: tl.constexpr,
):
    # Ensure TensorCore-friendly K tile
    tl.static_assert(BLOCK_K % 16 == 0)

    # Tile indices from 3D launch grid
    pid_m = tl.program_id(0)
    gid = tl.program_id(1) # group id
    pid_n = tl.program_id(2)

    # Tile offsets
    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M) # [BM]
    offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N) # [BN]
    mask_m = offs_m < M
    mask_n = offs_n < N

```

```

# Decompose M -> (batch_idx, out_pos)
batch_idx = offs_m // Lout # [BM]
out_pos = offs_m % Lout # [BM]
pos_base = out_pos * stride - padding # [BM]

# Group channel ranges for this gid
in_ch_start = gid * Cin_g
out_ch_start = gid * N

# Strides
batch_in_stride = Cin * Lin
batch_out_stride = Cout * Lout

# Precompute base pointers
batch_in_offs = batch_idx * batch_in_stride # [BM]
# Base for [BM, 1] rows at first input channel of this group
input_batch_group_base = input_ptr + batch_in_offs[:, None] + (in_ch_start * Lin)
out_ch_idx = out_ch_start + offs_n # [BN]

# Weight base column pointers: [1, BN]
w_col_base = weight_packed_ptr + out_ch_idx[None, :]

# Accumulator in fp32
acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)

mask_m_b = mask_m[:, None]
mask_n_b = mask_n[None, :]

# Loop over kernel taps then Cin_g in BLOCK_K chunks
# Important: hoist row validity outside K-loop to reduce mask materialization
for t in tl.static_range(0, kernel_size):
    tap_incr = t * dilation
    input_pos = pos_base[:, None] + tap_incr # [BM, 1]
    # Valid rows for this tap
    row_valid_mask = (input_pos >= 0) & (input_pos < Lin)
    row_valid_mask = row_valid_mask & mask_m_b # [BM, 1]

    # Double-buffer the (x, w) tiles across the Cin_g loop for better overlap
    # Prefetch the first chunk
    cin_off0 = 0 + tl.arange(0, BLOCK_K)
    k_mask0 = cin_off0 < Cin_g

    cin_incr0 = cin_off0 * Lin
    in_ptrs0 = input_batch_group_base + cin_incr0[None, :] + input_pos
    in_mask0 = row_valid_mask & k_mask0[None, :]
    x0 = tl.load(in_ptrs0, mask=in_mask0, other=0.0, cache_modifier=".ca")

    k_linear0 = cin_off0 * kernel_size + t
    w_ptrs0 = w_col_base + k_linear0[:, None] * Cout
    w_mask0 = k_mask0[:, None] & mask_n_b
    # Cache weights in L2 since the same Cout slice is reused for nearby tiles
    w0 = tl.load(w_ptrs0, mask=w_mask0, other=0.0, cache_modifier=".cg")

    # Process remaining chunks
    for cstart in tl.static_range(BLOCK_K, Cin_g, BLOCK_K):
        # Compute with current tiles
        acc = tl.dot(x0, w0, acc)

```

```

        # Prefetch next
        cin_off1 = cstart + tl.arange(0, BLOCK_K)
        k_mask1 = cin_off1 < Cin_g

        cin_incr1 = cin_off1 * Lin
        in_ptrs1 = input_batch_group_base + cin_incr1[None, :] + input_pos
        in_mask1 = row_valid_mask & k_mask1[None, :]
        x1 = tl.load(in_ptrs1, mask=in_mask1, other=0.0, cache_modifier=".ca")

        k_linear1 = cin_off1 * kernel_size + t
        w_ptrs1 = w_col_base + k_linear1[:, None] * Cout
        w_mask1 = k_mask1[:, None] & mask_n_b
        w1 = tl.load(w_ptrs1, mask=w_mask1, other=0.0, cache_modifier=".cg")

        # Swap buffers
        x0 = x1
        w0 = w1

        # Final dot for tail (or the only chunk if Cin_g <= BLOCK_K)
        acc = tl.dot(x0, w0, acc)

    # Bias add (fused epilogue)
    if has_bias:
        b = tl.load(bias_ptr + out_ch_idx, mask=mask_n, other=0.0).to(tl.float32)
        acc += b[None, :]

    # Store output [B, Cout, Lout] - cast to input dtype
    batch_out_offs = batch_idx * batch_out_stride
    out_ptrs = (
        output_ptr
        + batch_out_offs[:, None]
        + out_ch_idx[None, :] * Lout
        + out_pos[:, None]
    )
    out_mask = mask_m_b & mask_n_b
    # Cast accumulator back to input precision
    tl.store(out_ptrs, acc, mask=out_mask)

def _maybe_pack_weight(weight_tensor, in_channels_per_group, kernel_sz):
    """
    Packs weight from [Cout, Cin_g, Ksz] to [Cin_g*Ksz, Cout] using Triton.
    Caches the result based on data_ptr + shape + device to avoid repeated packing.
    Supports fp32, fp16, and bf16 dtypes.
    """
    Cout, Cin_g, Ksz = weight_tensor.shape
    assert Cin_g == in_channels_per_group and Ksz == kernel_sz

    # Check if caching is disabled
    if not _DISABLE_WEIGHT_CACHE:
        cache_key = (
            int(weight_tensor.data_ptr()),
            tuple(weight_tensor.shape),
            weight_tensor.device,
            weight_tensor.dtype,
        )
        packed = _WEIGHT_PACK_CACHE.get(cache_key, None)

```



```

    if (
        packed is not None
        and packed.is_cuda
        and packed.dtype == weight_tensor.dtype
    ):
        return packed

    # Allocate packed tensor with same dtype as input
    packed = torch.empty(
        (Cin_g * Ksz, Cout), device=weight_tensor.device, dtype=weight_tensor.dtype
    )

    # Launch Triton packing kernel
    BR, BC = 128, 128

    def grid(meta):
        R = Cin_g * Ksz
        C = Cout
        gr = (R + BR - 1) // BR
        gc = (C + BC - 1) // BC
        return (gr, gc)

    pack_conv1d_weight_kernel[grid](
        weight_tensor,
        packed,
        Cout,
        Cin_g,
        Ksz,
        BLOCK_R=BR,
        BLOCK_C=BC,
    )

    if not _DISABLE_WEIGHT_CACHE:
        _WEIGHT_PACK_CACHE[cache_key] = packed
    return packed

def kernel_function(
    input_tensor,
    weight_tensor,
    bias_tensor=None,
    stride=1,
    padding=1,
    dilation=1,
    groups=1,
):
    """
    Optimized Triton 1D convolution using GEMM tiling with packed weights.
    Supports fp32, fp16, and bf16 dtypes.
    """
    # Validate dtype consistency
    assert input_tensor.dtype in (
        torch.float32,
        torch.float16,
        torch.bfloat16,
    ), f"input_tensor must be fp32, fp16, or bf16, got {input_tensor.dtype}"
    assert (
        weight_tensor.dtype == input_tensor.dtype

```

```

), f"weight_tensor dtype {weight_tensor.dtype} must match input_tensor dtype {input_tensor.dtype}"
if bias_tensor is not None:
    assert (
        bias_tensor.dtype == input_tensor.dtype
    ), f"bias_tensor dtype {bias_tensor.dtype} must match input_tensor dtype {input_tensor.dtype}"

# Dimensions
batch_size, in_channels, input_length = input_tensor.shape
out_channels, in_channels_per_group, kernel_sz = weight_tensor.shape

# Validate inputs
assert in_channels_per_group * groups == in_channels, (
    f"in_channels_per_group ({in_channels_per_group}) * groups ({groups}) != "
    f"in_channels ({in_channels})"
)
assert (
    out_channels % groups == 0
), f"out_channels ({out_channels}) must be divisible by groups ({groups})"
out_channels_per_group = out_channels // groups

# Output length per PyTorch formula
output_length = (
    (input_length + 2 * padding - dilation * (kernel_sz - 1) - 1) // stride
) + 1

# Allocate output with same dtype as input
output = torch.empty(
    (batch_size, out_channels, output_length),
    device=input_tensor.device,
    dtype=input_tensor.dtype,
)

# Bias handling
has_bias = bias_tensor is not None
if not has_bias:
    bias_tensor = torch.zeros(
        out_channels, device=input_tensor.device, dtype=input_tensor.dtype
    )

# Pack weights once and cache
weight_packed = _maybe_pack_weight(weight_tensor, in_channels_per_group, kernel_sz)

# GEMM dims
M = batch_size * output_length
N = out_channels_per_group
K = in_channels_per_group * kernel_sz

# Grid calculation (pure Python)
def grid(meta):
    BM = meta["BLOCK_M"]
    BN = meta["BLOCK_N"]
    gm = (M + BM - 1) // BM
    gn = (N + BN - 1) // BN
    return (gm, groups, gn)

# Launch kernel
convld_gemm_kernel[grid](
    input_tensor,

```

```

        weight_packed,
        bias_tensor,
        output,
        batch_size,
        in_channels,
        out_channels,
        input_length,
        output_length,
        groups,
        M,
        N,
        K,
        in_channels_per_group, # Cin_g as constexpr
        kernel_sz,
        stride,
        padding,
        dilation,
        has_bias,
    )

    return output

class TritonModel(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, input_tensor, weight_tensor, bias_tensor):
        return kernel_function(
            input_tensor,
            weight_tensor,
            bias_tensor,
            stride=1,
            padding=1,
            dilation=1,
            groups=1,
        )

def get_inputs():
    """
    Generate test inputs for the conv1d benchmark.

    Returns:
        List of input tuples for benchmarking.
    """
    # Generate synthetic production-like data
    # Shape matches original production data: (2048, 96, 200)
    # Use torch.float32 to match production data dtype
    input_full, weight_tensor, bias_tensor = generate_synthetic_production_data(
        dtype=torch.float32, device="cuda"
    )

    print("Generated synthetic production data:")
    print(f" Input shape: {input_full.shape}, dtype: {input_full.dtype}")
    print(f" Weight shape: {weight_tensor.shape}, dtype: {weight_tensor.dtype}")
    print(f" Bias shape: {bias_tensor.shape}, dtype: {bias_tensor.dtype}")
    print(f" Input is contiguous: {input_full.is_contiguous()}")

```

```

# Create multiple batch sizes by slicing the full input
# This matches the original production data loading approach
batch_sizes = [64, 128, 256, 512, 1024, 2048]
inputs = []

for batch_size in batch_sizes:
    # Slice input to desired batch size
    input_tensor = input_full[:batch_size, :, :]
    if not input_tensor.is_contiguous():
        input_tensor = input_tensor.contiguous()
    inputs.append((input_tensor, weight_tensor, bias_tensor))
return inputs

# =====
# TritonBench Operator Integration
# =====

class Operator(BenchmarkOperator):
    """
    TritonBench operator for conv1d.

    This operator wraps the kernel_evaluate generated TritonModel and PytorchModel
    into the TritonBench evaluation framework.
    """
    DEFAULT_METRICS = ["latency"]
    DEFAULT_PRECISION = "fp32"

    def __init__(
        self,
        tb_args: argparse.Namespace,
        extra_args: Optional[List[str]] = None,
    ) -> None:
        super().__init__(tb_args, extra_args)

        self.pytorch_model = PytorchModel().to(self.device).eval()
        self.triton_model = TritonModel().to(self.device).eval()

    def get_input_iter(self) -> Generator:
        """
        Generate test inputs using the shared get_inputs() function.

        Note: get_inputs() is an independent function that both models access.
        This ensures consistent test data generation across all implementations.
        """
        inputs = get_inputs()

    def to_device_dtype(x):
        if isinstance(x, torch.Tensor):
            return x.to(device=self.device, dtype=self.dtype)
        return x

    for input_tuple in inputs:
        converted_tuple = tuple(to_device_dtype(tensor) for tensor in input_tuple)
        yield converted_tuple

```

```

def get_x_val(self, example_inputs: Any) -> str:
    """Extract x-axis value for plotting."""
    if not example_inputs or len(example_inputs) == 0:
        return "0"

    first_input = example_inputs[0]
    if isinstance(first_input, torch.Tensor):
        shape = first_input.shape

        if not shape:
            return "1"

        return "x".join(map(str, shape[: len(shape)]))

    return "Unknown"

@register_benchmark(baseline=True, operator_name="conv1d")
def pytorch_reference(self, *inputs) -> Callable:
    """
    PyTorch reference implementation (baseline).

    Uses the PytorchModel generated by kernel_evolve.
    """
    model = torch.compile(self.pytorch_model, mode="max-autotune-no-cudagraphs")

    def _impl():
        with torch.no_grad():
            return model(*inputs)

    return _impl

@register_benchmark(baseline=False, operator_name="conv1d")
def pytorch_reference_conv2d(self, *inputs) -> Callable:
    """
    PyTorch reference implementation (baseline).

    Uses the PytorchModel generated by kernel_evolve.
    """
    model = torch.compile(self.pytorch_model, mode="max-autotune-no-cudagraphs")

    def _impl():
        with torch.no_grad():
            return model(*inputs, use_conv2d=True)

    return _impl

@register_benchmark(operator_name="conv1d")
def triton_kernel(self, *inputs) -> Callable:
    """
    Triton kernel implementation.

    Uses the TritonModel generated by kernel_evolve.
    """
    model = torch.compile(self.triton_model, mode="max-autotune-no-cudagraphs")

    def _impl():
        with torch.no_grad():

```

```

        return model(*inputs)

    return _impl

# =====
# Evaluation Entry Point
# =====

def run():
    """Run benchmark with latency, speedup, and accuracy evaluation."""

    try:
        from tritonbench.utils.parser import get_parser

        override_args = [
            "--device",
            "cuda",
            "--mode",
            "fwd",
            "--metrics",
            "latency,speedup,accuracy",
            "--precision",
            "fp16",
            "--benchmark-name",
            "conv1d",
            "--atol",
            "1e-4",
            "--rtol",
            "5e-4",
        ]

        parser = get_parser()
        tb_args, extra_args = parser.parse_known_args(override_args)

        op = Operator(tb_args=tb_args, extra_args=extra_args)
        print("Running TritonBench conv1d benchmark...")
        op.run()

        # Print TritonBench output table
        print("\n" + str(op.output))

        # Also print formatted performance table with explicit headers
        if hasattr(op, "output") and op.output:
            print("\n" + "=" * 80)
            print("PERFORMANCE SUMMARY TABLE")
            print("=" * 80)

            ub_dict = op.output.userbenchmark_dict
            benchmark_name = op.output.benchmark_name
            x_vals = op.output.x_vals

            # Print table header
            print(
                f"{'x_val':<15} {'pytorch_reference-latency':<30} {'triton_kernel-latency':<30} {'triton_kernel-  

                speedup':<25}"
            )
    )

```



```

    print("-" * 100)

    # Print data rows
    for x_val in x_vals:
        latency_key = (
            f"tritonbench_{benchmark_name}[x_{x_val}-triton_kernel]_latency"
        )
        speedup_key = (
            f"tritonbench_{benchmark_name}[x_{x_val}-triton_kernel]_speedup"
        )
        ref_latency_key = (
            f"tritonbench_{benchmark_name}[x_{x_val}-pytorch_reference]_latency"
        )

        triton_latency = ub_dict.get(latency_key, "N/A")
        speedup = ub_dict.get(speedup_key, "N/A")
        ref_latency = ub_dict.get(ref_latency_key, "N/A")

        print(
            f"{x_val:<15} {str(ref_latency):<30} {str(triton_latency):<30} {str(speedup):<25}"
        )

    print("=" * 80 + "\n")
except Exception as e:
    print(f"ERROR: Test failed with exception: {e}")
    print("REFERENCE_TEST_FAILED")
    print("KERNEL_TEST_FAILED")
    print("FITNESS_SCORE: 0.0")
    return False

# Validate accuracy and report metrics
try:
    if not hasattr(op, "output") or not op.output:
        print("ERROR: No output available from benchmark")
        print("KERNEL_TEST_FAILED")
        print("FITNESS_SCORE: 0.0")
        return False

    ub_dict = op.output.userbenchmark_dict
    benchmark_name = op.output.benchmark_name
    x_vals = op.output.x_vals

    print("\n" + "=" * 80)
    print("DETAILED METRICS PER INPUT SIZE")
    print("=" * 80)

    accuracy_passed = True
    all_latencies = []
    all_speedups = []

    for x_val in x_vals:
        latency_key = (
            f"tritonbench_{benchmark_name}[x_{x_val}-triton_kernel]_latency"
        )
        speedup_key = (
            f"tritonbench_{benchmark_name}[x_{x_val}-triton_kernel]_speedup"
        )
        accuracy_key = (

```

```

        f"tritonbench_{benchmark_name}[x_{x_val}-triton_kernel]-accuracy"
    )

    latency = ub_dict.get(latency_key)
    speedup = ub_dict.get(speedup_key)
    accuracy = ub_dict.get(accuracy_key)

    print(f"\nInput size: {x_val}")
    print(f"    Latency: {latency}")
    print(f"    Speedup: {speedup}")
    print(f"    Accuracy: {accuracy}")

    if latency is not None:
        all_latencies.append(latency)
    if speedup is not None:
        all_speedups.append(speedup)

    if accuracy is not None and accuracy == 0.0:
        accuracy_passed = False
        print(f"    ACCURACY FAILED for input size {x_val}")

print("\n" + "=" * 80)
print("SUMMARY STATISTICS")
print("=" * 80)

avg_latency_key = f"tritonbench_{benchmark_name}[triton_kernel]-latency-avg"
avg_speedup_key = f"tritonbench_{benchmark_name}[triton_kernel]-speedup-avg"

avg_latency = ub_dict.get(avg_latency_key)
avg_speedup = ub_dict.get(avg_speedup_key)

print(f"Average Latency: {avg_latency}")
print(f"Average Speedup: {avg_speedup}")
print(
    f"Accuracy: {'PASS' if accuracy_passed else 'FAIL'} (all input sizes must pass)"
)
print("=" * 80 + "\n")

if not accuracy_passed:
    print(
        "ERROR: NUMERICAL MISMATCH BETWEEN TRITON KERNEL AND PYTORCH REFERENCE"
    )
    print("DEBUG: One or more input sizes failed accuracy check")
    print("KERNEL_TEST_FAILED")
    print("FITNESS_SCORE: 0.0")
    return False

print(
    "SUCCESS: Triton kernel output matches PyTorch reference implementation"
)
print("KERNEL_TEST_PASSED")

fitness_score = 1.0
if avg_speedup is not None and avg_speedup > 0:
    fitness_score = max(0.1, min(avg_speedup, 10.0))

print(f"FITNESS_SCORE: {fitness_score}")

```

```
        return True

    except Exception as e:
        print(f"ERROR: Test failed with exception: {e}")
        print("KERNEL_TEST_FAILED")
        print("FITNESS_SCORE: 0.0")
        return False

if __name__ == "__main__":
    success = run()
    sys.exit(0 if success else 1)
```